TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 7: BIỂU DIỄN TRI THỨC & SUY LUẬN LOGIC

Xây dựng hệ chuyên gia mini bằng luật IF-THEN

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày: 3/11/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả và dán vào khung viền xám đứt nét (🖼️)
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Giải thích được cách AI biểu diễn tri thức bằng luật IF-THEN
- Giải thích được cơ sở tri thức và luật suy diễn là gì
- Phân biệt được Forward Chaining và Backward Chaining
- Cài đặt được một hệ chuyên gia mini bằng Python
- Tự xây dựng được bộ luật 10–15 luật cho một lĩnh vực tự chọn

TIẾT 1 — TRẢI NGHIỆM: MÁY 'SUY LUẬN' NHƯ THẾ NÀO?

Mở notebook "Demo_Buoi7.ipynb" (đã có sẵn toàn bộ code bên dưới). Copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo: Hệ chuyên gia chẩn đoán bệnh dựa trên triệu chứng

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
# Cơ sở tri thức: các luật IF-THEN
luat_benh = [
    {"neu": ["sot", "ho", "dau_hong"], "thi": "cam_cum"},
    {"neu": ["hat_hoi", "ngua_mat", "chay_nuoc_mui"], "thi": "di_ung"},
    {"neu": ["sot", "phat_ban", "dau_dau"], "thi": "sot_xuat_huyet"},
]

def suy_dien(trieu_chung_nhap, luat):
    ket_luan = []
    for r in luat:
        if all(tc in trieu_chung_nhap for tc in r["neu"]):
            ket_luan.append(r["thi"])
    return ket_luan

trieu_chung = ["sot", "ho", "dau_hong"]
ket_qua = suy_dien(trieu_chung, luat_benh)
print("Triệu chứng nhập vào:", trieu_chung)
print("Chẩn đoán:", ket_qua if ket_qua else "Không xác định được bệnh")
```

💭 **Suy nghĩ: Vì sao hệ thống chẩn đoán ra 'cam_cum' với bộ triệu chứng này? Nó có thực sự 'hiểu' về y khoa không?**

Hệ thống chẩn đoán ra `cam_cum` vì bộ triệu chứng đầu vào (sốt, ho, đau họng) khớp hoàn toàn với điều kiện của luật IF-THEN đã được lập trình sẵn.

Hệ thống không thực sự hiểu về y khoa mà chỉ đối chiếu triệu chứng với các quy tắc trong cơ sở tri thức để đưa ra kết luận. Nó không có khả năng tự phân tích hay tự học như con người.

🔧 Cải tiến: Hãy thử 3 bộ triệu chứng khác nhau — (1) khớp đúng luật dị ứng, (2) chỉ khớp MỘT PHẦN một luật (thiếu 1 triệu chứng), (3) hoàn toàn không khớp luật nào — rồi in kết quả từng trường hợp:

```
# 1. Khớp đúng luật dị ứng
trieu_chung_1 = ["hat_hoi", "ngua_mat", "chay_nuoc_mui"]

# 2. Chỉ khớp một phần luật cảm cúm (thiếu đau họng)
trieu_chung_2 = ["sot", "ho"]

# 3. Hoàn toàn không khớp luật nào
trieu_chung_3 = ["dau_bung", "chong_mat"]

# Kiểm tra và in kết quả
for i, trieu_chung in enumerate(
    [trieu_chung_1, trieu_chung_2, trieu_chung_3], 1
):
    ket_qua = suy_dien(trieu_chung, luat_benh)

    print(f"\nTrường hợp {i}:")
    print("Triệu chứng:", trieu_chung)
    print("Kết quả:", ket_qua if ket_qua else "Không xác định được bệnh")
```

🌾 Thu hoạch: chụp màn hình kết quả 3 trường hợp thử nghiệm:

```
        [trieu_chung_1, trieu_chung_2, trieu_chung_3], 1
    ):
        ket_qua = suy_dien(trieu_chung, luat_benh)

        print(f"\nTrường hợp {i}:")
        print("Triệu chứng:", trieu_chung)
        print("Kết quả:", ket_qua if ket_qua else "Không xác định được bệnh")
```

```
...
Trường hợp 1:
Triệu chứng: ['hat_hoi', 'ngua_mat', 'chay_nuoc_mui']
Kết quả: ['di_ung']

Trường hợp 2:
Triệu chứng: ['sot', 'ho']
Kết quả: Không xác định được bệnh

Trường hợp 3:
Triệu chứng: ['dau_bung', 'chong_mat']
Kết quả: Không xác định được bệnh
```

Câu hỏi thảo luận: Nếu muốn hệ thống chẩn đoán 'thông minh' hơn (không chỉ so khớp từ khóa), cần thêm điều gì?

Để hệ thống chẩn đoán thông minh hơn, cần bổ sung:

1. Cơ sở tri thức phong phú: Thêm nhiều bệnh và triệu chứng, bao gồm mức độ và mối liên hệ giữa chúng.
2. Suy luận xác suất: Đánh giá khả năng mắc bệnh thay vì chỉ so khớp chính xác các triệu chứng.
3. Machine Learning (Học máy): Cho phép hệ thống học từ dữ liệu và cải thiện khả năng dự đoán.
4. Xử lý ngôn ngữ tự nhiên (NLP): Hiểu triệu chứng được người dùng mô tả bằng ngôn ngữ tự nhiên.

Kết luận: Cần kết hợp cơ sở tri thức, suy luận xác suất và AI/Machine Learning để hệ thống có khả năng phân tích linh hoạt hơn, thay vì chỉ phụ thuộc vào các luật IF-THEN cố định.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

Logic mệnh đề AND/OR/NOT/IF-THEN nghĩa là gì? Trong code Demo, phần nào tương ứng với AND?

Đây là các phép logic dùng để biểu diễn điều kiện và đưa ra kết luận:

- AND (VÀ): Tất cả điều kiện phải đúng.
- OR (HOẶC): Chỉ cần một trong các điều kiện đúng.
- NOT (PHỦ ĐỊNH): Điều kiện không đúng hoặc không tồn tại.
- IF-THEN (NẾU – THÌ): Nếu điều kiện thỏa mãn thì đưa ra kết luận.

Trong code Demo, phần tương ứng với AND là:

```
None
if all(tc in trieu_chung_nhap for tc in r["neu"]):
```

Hàm `all()` kiểm tra tất cả triệu chứng trong luật có xuất hiện trong danh sách nhập vào hay không. Chỉ khi tất cả đều đúng thì hệ thống mới đưa ra kết luận.

Cơ sở tri thức (Knowledge Base) và luật suy diễn là gì?

Cơ sở tri thức (Knowledge Base): Là nơi lưu trữ các kiến thức, thông tin và quy tắc về một lĩnh vực cụ thể.

- Luật suy diễn: Là những quy tắc IF-THEN dùng để suy luận từ các dữ liệu đã biết để đưa ra kết luận mới.

Ví dụ:

```
None
{"neu": ["sot", "ho", "dau_hong"], "thi": "cam_cum"}
```

Trong đó:

- Knowledge Base chứa luật chẩn đoán bệnh.
- Luật suy diễn quy định rằng nếu có đủ ba triệu chứng sốt, ho và đau họng thì kết luận là cảm cúm.

Forward Chaining khác Backward Chaining như thế nào? Cho một ví dụ đời sống cho mỗi loại.

Tiêu chí	Forward Chaining	Backward Chaining
Cách suy luận	Từ dữ kiện đến kết luận	Từ kết luận cần tìm đến dữ kiện
Hướng suy luận	Tiến	Lùi
Bắt đầu từ	Dữ liệu đã biết	Mục tiêu cần chứng minh

Ví dụ Forward Chaining (Suy diễn tiến):

- Biết trời đang mưa.
- Nếu trời mưa thì đường sẽ ướt.
- Kết luận: Đường đang ướt.

Ví dụ Backward Chaining (Suy diễn lùi):

- Muốn kiểm tra đường có ướt không.
- Biết rằng nếu trời mưa thì đường sẽ ướt.
- Kiểm tra xem trời có đang mưa không.
- Nếu có, kết luận đường có thể đang ướt.

Vì sao đôi khi một luật cần được áp dụng NHIỀU LẦN (suy diễn dây chuyền) thay vì chỉ một lần?

Vì kết luận của luật trước có thể trở thành dữ kiện đầu vào cho luật tiếp theo. Do đó, hệ thống phải tiếp tục suy diễn cho đến khi tìm được kết luận cuối cùng hoặc không còn luật nào phù hợp.

Ví dụ:

- Luật 1: Nếu trời mưa → Đường ướt.
- Luật 2: Nếu đường ướt → Đường trơn.
- Luật 3: Nếu đường trơn → Cần đi chậm.

Từ dữ kiện ban đầu là trời mưa, hệ thống phải áp dụng liên tiếp ba luật để kết luận cần đi chậm.

Kết luận: Suy diễn dây chuyền giúp hệ chuyên gia giải quyết các vấn đề phức tạp bằng cách sử dụng kết quả trung gian để tiếp tục tìm ra kết luận mới, thay vì chỉ áp dụng một luật duy nhất.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: HỆ CHUYÊN GIA TƯ VẤN CÂY TRỒNG

📋 Nội dung có sẵn — hệ chuyên gia tư vấn chọn cây trồng (một bước). Copy và chạy:

```
luat_cay_trong = [
    {"neu": ["dat_chua", "khi_hau_nong"], "thi": "trong_dua_hau"},
    {"neu": ["dat_phi_nhieu", "nhieu_nuoc"], "thi": "trong_lua"},
            {"neu":   ["dat_kho",   "it_nuoc",   "khi_hau_nong"],   "thi":
"trong_xuong_rong"},
    {"neu": ["dat_phi_nhieu", "khi_hau_mat"], "thi": "trong_rau_cai"},
]

def suy_dien(dieu_kien, luat):
    ket_luan = []
    for r in luat:
        if all(dk in dieu_kien for dk in r["neu"]):
            ket_luan.append(r["thi"])
    return ket_luan

dieu_kien_dat = ["dat_chua", "khi_hau_nong"]
ket_qua = suy_dien(dieu_kien_dat, luat_cay_trong)
print("Điều kiện:", dieu_kien_dat)
print("Gợi ý cây trồng:", ket_qua if ket_qua else "Chưa có gợi ý phù hợp")
```

💭 **Suy nghĩ: Nếu dieu_kien_dat chỉ có ["dat_phi_nhieu"] (thiếu "nhieu_nuoc"), kết quả sẽ ra sao? Vì sao?**

Kết quả sẽ là:

```
None
Điều kiện: ['dat_phi_nhieu']
Gợi ý cây trồng: Chưa có gợi ý phù hợp
```

Giải thích: Vì luật trồng lúa yêu cầu đồng thời hai điều kiện `dat_phi_nhieu` AND `nhieu_nuoc`. Khi chỉ có `dat_phi_nhieu`, hệ thống không thỏa mãn toàn bộ điều kiện nên không đưa ra kết luận `trong_lua`.

Kết luận: Hệ thống sử dụng phép logic AND, yêu cầu tất cả điều kiện trong một luật phải đúng thì mới đưa ra kết quả.

🔧 Cải tiến: Hãy thêm ít nhất 2 luật mới vào luat_cay_trong (VD: cây trồng phù hợp đất mặn, đất đồi núi…) và thử với điều kiện mới:

```
luat_cay_trong = [
```

```
    {"neu": ["dat_chua", "khi_hau_nong"], "thi": "trong_dua_hau"},
    {"neu": ["dat_phi_nhieu", "nhieu_nuoc"], "thi": "trong_lua"},
    {"neu": ["dat_kho", "it_nuoc", "khi_hau_nong"], "thi": "trong_xuong_rong"},
    {"neu": ["dat_phi_nhieu", "khi_hau_mat"], "thi": "trong_rau_cai"},

    # Hai luật mới
    {"neu": ["dat_man", "khi_hau_nong"], "thi": "trong_dua"},
    {"neu": ["dat_doi_nui", "dat_kho"], "thi": "trong_che"},
]

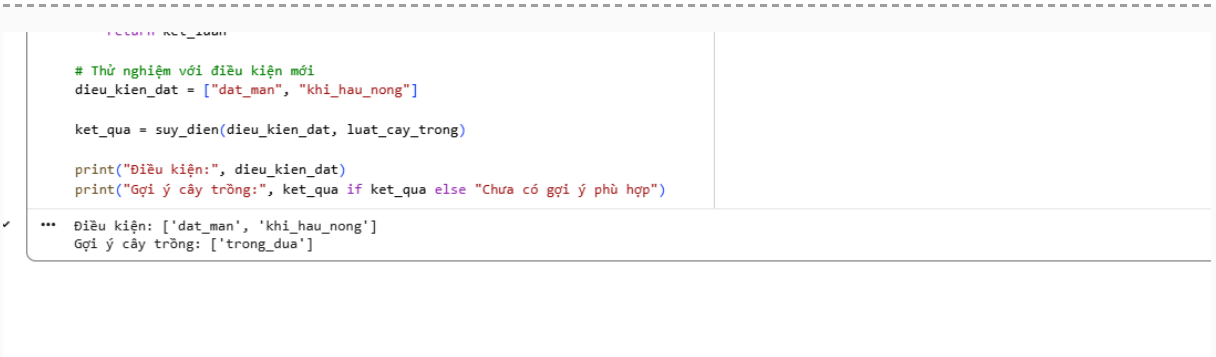
def suy_dien(dieu_kien, luat):
    ket_luan = []
    for r in luat:
        if all(dk in dieu_kien for dk in r["neu"]):
            ket_luan.append(r["thi"])
    return ket_luan

# Thử nghiệm với điều kiện mới
dieu_kien_dat = ["dat_man", "khi_hau_nong"]

ket_qua = suy_dien(dieu_kien_dat, luat_cay_trong)

print("Điều kiện:", dieu_kien_dat)
print("Gợi ý cây trồng:", ket_qua if ket_qua else "Chưa có gợi ý phù hợp")
```

🌾 Thu hoạch: chụp màn hình kết quả sau khi thêm luật mới:



📋 Nội dung có sẵn — suy diễn tiến nhiều bước (Forward Chaining thật sự). Copy và chạy:

```
luat_nhieu_buoc = [
    {"neu": ["dat_phi_nhieu", "nhieu_nuoc"], "thi": "phu_hop_lua_nuoc"},
                {"neu":    ["phu_hop_lua_nuoc",    "co_may_moc"],    "thi":
"nen_trong_lua_quy_mo_lon"},
]

def forward_chaining(dieu_kien_ban_dau, luat):
    facts = set(dieu_kien_ban_dau)
    thay_doi = True
    while thay_doi:
        thay_doi = False
        for r in luat:
            if all(dk in facts for dk in r["neu"]) and r["thi"] not in facts:
                facts.add(r["thi"])
                thay_doi = True
    return facts

dieu_kien = ["dat_phi_nhieu", "nhieu_nuoc", "co_may_moc"]
ket_qua_suy_dien = forward_chaining(dieu_kien, luat_nhieu_buoc)
```

```
print("Tất cả sự kiện suy ra được:", ket_qua_suy_dien)
```

💭 **Suy nghĩ: forward_chaining() khác suy_dien() ở Hoạt động trước ở điểm nào? Vì sao cần vòng lặp while thay vì chỉ duyệt luật một lần?**

1. `forward_chaining()` khác `suy_dien()` ở điểm nào?

- `suy_dien()`: Chỉ kiểm tra các điều kiện ban đầu với từng luật và đưa ra kết luận trực tiếp. Không sử dụng kết luận mới để suy diễn tiếp.
- `forward_chaining()`: Có khả năng suy diễn nhiều bước. Kết luận từ luật trước được thêm vào tập sự kiện và trở thành dữ kiện để áp dụng luật tiếp theo.

2. Vì sao cần vòng lặp `while` thay vì chỉ duyệt luật một lần?

Vì một kết luận mới có thể kích hoạt thêm luật khác. Vòng lặp `while` giúp hệ thống tiếp tục kiểm tra và suy diễn cho đến khi không còn sự kiện mới nào được tạo ra.

3. Ví dụ trong code

- Ban đầu: Đất phì nhiêu, nhiều nước, có máy móc.
- Luật 1: Đất phì nhiêu + nhiều nước → Phù hợp lúa nước.
- Luật 2: Phù hợp lúa nước + có máy móc → Nên trồng lúa quy mô lớn.

Kết quả cuối cùng: `nen_trong_lua_quy_mo_lon`.

Kết luận: `suy_dien()` chỉ suy luận trực tiếp một bước, còn `forward_chaining()` có thể suy luận dây chuyền nhiều bước nhờ vòng lặp `while`, giúp hệ thống đưa ra những kết luận phức tạp hơn.

🔧 Cải tiến: Hãy thêm một luật thứ 3 vào luat_nhieu_buoc sao cho kết luận của luật 2 ("nen_trong_lua_quy_mo_lon") lại kích hoạt tiếp một luật mới (suy diễn 3 tầng):

```
luat_nhieu_buoc = [
    {"neu": ["dat_phi_nhieu", "nhieu_nuoc"],
     "thi": "phu_hop_lua_nuoc"},

    {"neu": ["phu_hop_lua_nuoc", "co_may_moc"],
     "thi": "nen_trong_lua_quy_mo_lon"},

    # Luật thứ 3
    {"neu": ["nen_trong_lua_quy_mo_lon"],
     "thi": "can_dau_tu_he_thong_tuoi_tieu"},
]

def forward_chaining(dieu_kien_ban_dau, luat):
    facts = set(dieu_kien_ban_dau)
    thay_doi = True

    while thay_doi:
        thay_doi = False

        for r in luat:
            if all(dk in facts for dk in r["neu"]) and r["thi"] not in facts:
                facts.add(r["thi"])
                thay_doi = True

    return facts
```

```
dieu_kien = ["dat_phi_nhieu", "nhieu_nuoc", "co_may_moc"]

ket_qua_suy_dien = forward_chaining(dieu_kien, luat_nhieu_buoc)

print("Tất cả sự kiện suy ra được:", ket_qua_suy_dien)
```

🌾 Thu hoạch: chụp màn hình tất cả sự kiện suy ra được sau khi thêm luật tầng 3:



TIẾT 4 — ÁP DỤNG MỞ RỘNG: TỰ XÂY HỆ CHUYÊN GIA RIÊNG

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy code mẫu hệ chuyên gia tư vấn chọn ngành học (dùng lại hàm forward_chaining ở Tiết 3):

```
luat_chon_nganh = [
    {"neu": ["thich_toan", "thich_logic"], "thi": "phu_hop_cntt"},
    {"neu": ["thich_ve", "thich_sang_tao"], "thi": "phu_hop_thiet_ke"},
              {"neu":    ["thich_giao_tiep",    "thich_kinh_doanh"],    "thi":
"phu_hop_marketing"},
          {"neu":   ["thich_sinh_hoc",   "thich_giup_do_nguoi_khac"],   "thi":
"phu_hop_y_duoc"},
                  {"neu":     ["phu_hop_cntt",     "gioi_tieng_anh"],     "thi":
"nen_hoc_cntt_quoc_te"},
]

so_thich = ["thich_toan", "thich_logic", "gioi_tieng_anh"]
ket_qua = forward_chaining(so_thich, luat_chon_nganh)
print("Các sự kiện suy luận được:", ket_qua)
```

☐ Chạy code trên và thử với ít nhất 2 bộ sở thích khác nhau

🔧 Cải tiến: Cả nhóm chọn MỘT lĩnh vực bất kỳ (giữ nguyên 'chọn ngành học' hoặc đổi hoàn toàn sang lĩnh vực khác — tư vấn dinh dưỡng, tư vấn chọn thiết bị, tư vấn chọn môn thể thao…), tự viết 10–15 luật cho lĩnh vực đó, và thử nghiệm với ít nhất 3 bộ dữ kiện đầu vào khác nhau:

```
# ========================================
# HỆ CHUYÊN GIA TƯ VẤN CHỌN NGÀNH HỌC
# Sử dụng phương pháp Forward Chaining
# ========================================

# 1. XÂY DỰNG CƠ SỞ TRI THỨC (12 LUẬT IF-THEN)

luat_chon_nganh = [
   # Nhóm Công nghệ thông tin
   {"neu": ["thich_toan", "thich_logic"], "thi": "phu_hop_cntt"},
   {"neu": ["thich_lap_trinh", "thich_cong_nghe"], "thi": "phu_hop_cntt"},
   {"neu": ["phu_hop_cntt", "gioi_tieng_anh"], "thi": "nen_hoc_cntt_quoc_te"},
```

```
    # Nhóm Thiết kế
    {"neu": ["thich_ve", "thich_sang_tao"], "thi": "phu_hop_thiet_ke"},
    {"neu": ["phu_hop_thiet_ke", "biet_su_dung_phan_mem"], "thi": "nen_hoc_thiet_ke_do_hoa"},

    # Nhóm Marketing
    {"neu": ["thich_giao_tiep", "thich_kinh_doanh"], "thi": "phu_hop_marketing"},
    {"neu": ["phu_hop_marketing", "thich_sang_tao"], "thi": "nen_hoc_truyen_thong"},

    # Nhóm Y dược
    {"neu": ["thich_sinh_hoc", "thich_giup_do_nguoi_khac"], "thi": "phu_hop_y_duoc"},
    {"neu": ["phu_hop_y_duoc", "hoc_tot_hoa_hoc"], "thi": "nen_hoc_duoc"},

    # Nhóm Kinh tế
    {"neu": ["thich_tinh_toan", "thich_phan_tich"], "thi": "phu_hop_kinh_te"},
    {"neu": ["phu_hop_kinh_te", "thich_kinh_doanh"], "thi": "nen_hoc_quan_tri_kinh_doanh"},

    # Nhóm Ngôn ngữ
    {"neu": ["gioi_tieng_anh", "thich_giao_tiep"], "thi": "phu_hop_ngon_ngu"},
]


# 2. XÂY DỰNG HÀM SUY DIỄN TIẾN

def forward_chaining(dieu_kien_ban_dau, luat):
    facts = set(dieu_kien_ban_dau)
    thay_doi = True

    while thay_doi:
        thay_doi = False

        for r in luat:
            if all(dk in facts for dk in r["neu"]) and r["thi"] not in facts:
                facts.add(r["thi"])
                thay_doi = True

    return facts


# 3. XÂY DỰNG 3 BỘ DỮ KIỆN THỬ NGHIỆM

bo_du_lieu = [
    {
        "ten": "Trường hợp 1 - Người thích công nghệ",
        "dieu_kien": [
            "thich_toan",
            "thich_logic",
            "gioi_tieng_anh"
        ]
    },
    {
        "ten": "Trường hợp 2 - Người thích thiết kế",
        "dieu_kien": [
            "thich_ve",
            "thich_sang_tao",
            "biet_su_dung_phan_mem"
        ]
    },
    {
        "ten": "Trường hợp 3 - Người thích giao tiếp và kinh doanh",
```

```
        "dieu_kien": [
            "thich_giao_tiep",
            "thich_kinh_doanh",
            "thich_sang_tao"
        ]
    }
]


# 4. CHẠY THỬ NGHIỆM VÀ IN KẾT QUẢ

print("=" * 60)
print("HỆ CHUYÊN GIA TƯ VẤN CHỌN NGÀNH HỌC")
print("Phương pháp: FORWARD CHAINING")
print("Số lượng luật:", len(luat_chon_nganh))
print("=" * 60)

for truong_hop in bo_du_lieu:
    print("\n" + "-" * 60)
    print(truong_hop["ten"])

    print("Dữ kiện đầu vào:", truong_hop["dieu_kien"])

    ket_qua = forward_chaining(
        truong_hop["dieu_kien"],
        luat_chon_nganh
    )

    # Chỉ hiển thị các kết luận được suy diễn thêm
    ket_luan = ket_qua - set(truong_hop["dieu_kien"])

    print("Các sự kiện suy luận được:")

    if ket_luan:
        for su_kien in sorted(ket_luan):
            print("-", su_kien)
    else:
        print("Không có kết luận phù hợp.")

print("\n" + "=" * 60)
print("HOÀN THÀNH 3 TRƯỜNG HỢP THỬ NGHIỆM")
print("=" * 60)
```

🌾 Thu hoạch: chụp màn hình kết quả thử nghiệm hệ chuyên gia của nhóm:

```
============================================================
HỆ CHUYÊN GIA TƯ VẤN CHỌN NGÀNH HỌC
Phương pháp: FORWARD CHAINING
Số lượng luật: 12
============================================================

------------------------------------------------------------
Trường hợp 1 - Người thích công nghệ
Dữ kiện đầu vào: ['thich_toan', 'thich_logic', 'gioi_tieng_anh']
Các sự kiện suy luận được:
- nen_hoc_cntt_quoc_te
- phu_hop_cntt

------------------------------------------------------------
Trường hợp 2 - Người thích thiết kế
Dữ kiện đầu vào: ['thich_ve', 'thich_sang_tao', 'biet_su_dung_phan_mem']
Các sự kiện suy luận được:
- nen_hoc_thiet_ke_do_hoa
- phu_hop_thiet_ke

------------------------------------------------------------
Trường hợp 3 - Người thích giao tiếp và kinh doanh
Dữ kiện đầu vào: ['thich_giao_tiep', 'thich_kinh_doanh', 'thich_sang_tao']
Các sự kiện suy luận được:
- nen_hoc_truyen_thong
- phu_hop_marketing

============================================================
HOÀN THÀNH 3 TRƯỜNG HỢP THỬ NGHIỆM
============================================================
```

Nhận xét của nhóm: lĩnh vực nhóm chọn là gì? Kết luận thú vị nhất hệ thống suy ra được là gì?

Lĩnh vực nhóm chọn: Tư vấn chọn ngành học dựa trên sở thích, năng lực và định hướng của người học.

Kết luận thú vị nhất: Khi người học thích Toán, thích Logic và giỏi tiếng Anh, hệ thống suy luận được rằng người đó phù hợp với ngành Công nghệ thông tin và nên học CNTT quốc tế.

Giải thích: Hệ thống không chỉ đưa ra kết luận trực tiếp mà còn sử dụng kết quả của luật trước để kích hoạt luật tiếp theo (suy diễn nhiều tầng bằng Forward Chaining).

TIẾT 4 (BỔ SUNG) — 5 BÀI THỰC HÀNH THÊM DÀNH CHO SINH VIÊN CNTT

Năm bài dưới đây áp dụng hệ chuyên gia và suy diễn bằng luật vào các bài toán quen thuộc của ngành CNTT: xử lý sự cố mạng, bảo mật/phân quyền, lọc thư rác và phân tích thuật toán. Mỗi bài đều có code chạy sẵn trong notebook Thuchanh_Buoi7.ipynb: bạn chỉ cần copy/paste → chạy → suy nghĩ → cải tiến → thu hoạch. Các bài độc lập, có thể làm theo thứ tự bất kỳ.

Bài 1 — Hệ chuyên gia chẩn đoán sự cố mạng (có khả năng giải thích)

Liên hệ CNTT: đây là việc kỹ thuật viên helpdesk làm hằng ngày (kiểm tra cáp → IP → gateway → Internet → DNS). Các hệ thống AIOps/NOC thật cũng mã hóa kinh nghiệm này thành luật. Điểm quan trọng: hệ chuyên gia phải GIẢI THÍCH được vì sao nó kết luận như vậy (explanation facility).

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
luat_mang = [
    {"ten": "R1",  "neu": ["den_link_tat"], "thi": "loi_lop_vat_ly"},
    {"ten": "R2",  "neu": ["ip_dang_169_254"], "thi": "khong_nhan_duoc_dhcp"},
            {"ten":   "R3",      "neu":   ["khong_nhan_duoc_dhcp",   "day_cap_ok"],   "thi":
"nghi_loi_dhcp_server"},
    {"ten": "R4",  "neu": ["co_ip_hop_le", "ping_gateway_that_bai"], "thi": "loi_mang_noi_bo"},
           {"ten":   "R5",      "neu":   ["ping_gateway_ok",   "ping_8_8_8_8_that_bai"],   "thi":
"loi_ra_internet"},
    {"ten": "R6",  "neu": ["ping_8_8_8_8_ok", "phan_giai_ten_mien_that_bai"], "thi": "loi_dns"},
    {"ten": "R7",  "neu": ["loi_dns"], "thi": "de_xuat_doi_dns_sang_8_8_8_8"},
    {"ten": "R8",  "neu": ["loi_lop_vat_ly"], "thi": "de_xuat_kiem_tra_day_cap_va_card_mang"},
                    {"ten":     "R9",          "neu":     ["nghi_loi_dhcp_server"],     "thi":
"de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp"},
    {"ten": "R10", "neu": ["loi_mang_noi_bo"], "thi": "de_xuat_kiem_tra_switch_va_gateway"},
    {"ten": "R11", "neu": ["loi_ra_internet"], "thi": "de_xuat_lien_he_nha_mang_isp"},
]

def forward_chaining_co_giai_thich(su_kien, luat):
    """Trả về {sự_kiện: lý_do}. Sự kiện ban đầu có lý do là 'quan sát'; sự kiện suy ra ghi lại
luật đã sinh ra nó."""
    facts = {f: "quan sát" for f in su_kien}
    thay_doi = True
    while thay_doi:
        thay_doi = False
        for r in luat:
            if r["thi"] not in facts and all(dk in facts for dk in r["neu"]):
                facts[r["thi"]] = f"{r['ten']}: NẾU {' VÀ '.join(r['neu'])} THÌ {r['thi']}"
                thay_doi = True
    return facts

def chan_doan(ten_ca, su_kien):
    facts = forward_chaining_co_giai_thich(su_kien, luat_mang)
    suy_ra = {k: v for k, v in facts.items() if v != "quan sát"}
    print(f"=== {ten_ca} ===")
    print("Quan sát:", su_kien)
    if not suy_ra:
        print("-> Chưa đủ dữ kiện để kết luận. Hãy kiểm tra thêm.\n")
        return
    for k, ly_do in suy_ra.items():
        print(f"  * {k}   <= {ly_do}")
    print()

chan_doan("Ca 1: mở được IP nhưng không vào được web theo tên miền",
          ["co_ip_hop_le", "ping_gateway_ok", "ping_8_8_8_8_ok", "phan_giai_ten_mien_that_bai"])
chan_doan("Ca 2: laptop mới cắm cáp, IP bắt đầu bằng 169.254",
          ["ip_dang_169_254", "day_cap_ok"])
chan_doan("Ca 3: ping gateway thành công nhưng ra Internet thì không",
          ["co_ip_hop_le", "ping_gateway_ok", "ping_8_8_8_8_that_bai"])
chan_doan("Ca 4: chỉ biết máy có IP hợp lệ",
          ["co_ip_hop_le"])
```

```
=== Ca 1: mở được IP nhưng không vào được web theo tên miền ===
Quan sát: ['co_ip_hop_le', 'ping_gateway_ok', 'ping_8_8_8_8_ok', 'phan_giai_ten_mien_that_bai']
  * loi_dns   <= R6: NẾU ping_8_8_8_8_ok VÀ phan_giai_ten_mien_that_bai THÌ loi_dns
  * de_xuat_doi_dns_sang_8_8_8_8   <= R7: NẾU loi_dns THÌ de_xuat_doi_dns_sang_8_8_8_8

=== Ca 2: laptop mới cắm cáp, IP bắt đầu bằng 169.254 ===
Quan sát: ['ip_dang_169_254', 'day_cap_ok']
  * khong_nhan_duoc_dhcp   <= R2: NẾU ip_dang_169_254 THÌ khong_nhan_duoc_dhcp
  * nghi_loi_dhcp_server   <= R3: NẾU khong_nhan_duoc_dhcp VÀ day_cap_ok THÌ nghi_loi_dhcp_server
  * de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp   <= R9: NẾU nghi_loi_dhcp_server THÌ de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp

=== Ca 3: ping gateway thành công nhưng ra Internet thì không ===
Quan sát: ['co_ip_hop_le', 'ping_gateway_ok', 'ping_8_8_8_8_that_bai']
  * loi_ra_internet   <= R5: NẾU ping_gateway_ok VÀ ping_8_8_8_8_that_bai THÌ loi_ra_internet
  * de_xuat_lien_he_nha_mang_isp   <= R11: NẾU loi_ra_internet THÌ de_xuat_lien_he_nha_mang_isp

=== Ca 4: chỉ biết máy có IP hợp lệ ===
Quan sát: ['co_ip_hop_le']
-> Chưa đủ dữ kiện để kết luận. Hãy kiểm tra thêm.
```

💭 **Suy nghĩ: Ở Ca 1, chuỗi suy luận đi qua những luật nào? Vì sao việc ghi lại "luật nào sinh ra kết luận" lại quan trọng với người dùng hệ chuyên gia (gợi ý: niềm tin, gỡ lỗi bộ luật)? Ở Ca 4, hệ thống đưa ra kết quả gì và hành vi đó có hợp lý không?**

Nhận xét từng trường hợp:

1. Ca 1: Hệ thống áp dụng luật R6 và R7, xác định lỗi DNS và đề xuất đổi DNS sang `8.8.8.8`.
2. Ca 2: Hệ thống áp dụng R2, R3 và R9, xác định nghi ngờ lỗi DHCP Server và đề xuất kiểm tra DHCP hoặc đặt IP tĩnh.
3. Ca 3: Hệ thống áp dụng R5 và R11, xác định lỗi kết nối Internet và đề xuất liên hệ nhà mạng ISP.
4. Ca 4: Hệ thống không đưa ra kết luận vì chỉ có thông tin IP hợp lệ, chưa đủ dữ kiện để xác định nguyên nhân.

Kết luận: Hệ chuyên gia có khả năng suy diễn nhiều tầng bằng Forward Chaining, giải thích được luật nào tạo ra kết luận và biết từ chối đưa ra chẩn đoán khi thiếu dữ kiện.

Điểm đáng chú ý là việc lưu lại luật suy diễn giúp người dùng hiểu lý do hệ thống đưa ra kết quả, tăng tính minh bạch và hỗ trợ phát hiện lỗi trong cơ sở tri thức.

🔧 Cải tiến: Thêm 3 luật mới của riêng bạn (ví dụ: lỗi proxy/trình duyệt khi ping 8.8.8.8 và phân giải tên miền đều tốt nhưng web không mở; lỗi Wi-Fi sai mật khẩu; xung đột địa chỉ IP) rồi viết thêm 2 ca chẩn đoán mới để kiểm tra chúng.

```
# ========================================

# HE CHUYEN GIA CHAN DOAN LOI MANG

# SU DUNG FORWARD CHAINING CO GIAI THICH

# ========================================


# 1. CO SO TRI THUC: 14 LUAT IF-THEN


luat_mang = [

  {"ten": "R1", "neu": ["den_link_tat"], "thi": "loi_lop_vat_ly"},

  {"ten": "R2", "neu": ["ip_dang_169_254"], "thi": "khong_nhan_duoc_dhcp"},

  {"ten": "R3", "neu": ["khong_nhan_duoc_dhcp", "day_cap_ok"], "thi": "nghi_loi_dhcp_server"},

  {"ten": "R4", "neu": ["co_ip_hop_le", "ping_gateway_that_bai"], "thi": "loi_mang_noi_bo"},

  {"ten": "R5", "neu": ["ping_gateway_ok", "ping_8_8_8_8_that_bai"], "thi": "loi_ra_internet"},

  {"ten": "R6", "neu": ["ping_8_8_8_8_ok", "phan_giai_ten_mien_that_bai"], "thi": "loi_dns"},

  {"ten": "R7", "neu": ["loi_dns"], "thi": "de_xuat_doi_dns_sang_8_8_8_8"},

  {"ten": "R8", "neu": ["loi_lop_vat_ly"], "thi": "de_xuat_kiem_tra_day_cap_va_card_mang"},

  {"ten": "R9", "neu": ["nghi_loi_dhcp_server"], "thi": "de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp"},
```

```
    {"ten": "R10", "neu": ["loi_mang_noi_bo"], "thi": "de_xuat_kiem_tra_switch_va_gateway"},
    {"ten": "R11", "neu": ["loi_ra_internet"], "thi": "de_xuat_lien_he_nha_mang_isp"},

    # 3 LUAT MOI
    {"ten": "R12",
     "neu": ["ping_8_8_8_8_ok", "phan_giai_ten_mien_ok", "web_khong_mo"],
     "thi": "nghi_loi_proxy_hoac_trinh_duyet"},

    {"ten": "R13",
     "neu": ["wifi_xac_thuc_that_bai"],
     "thi": "nghi_sai_mat_khau_wifi"},

    {"ten": "R14",
     "neu": ["phat_hien_trung_ip"],
     "thi": "nghi_xung_dot_dia_chi_ip"}
]



# 2. HAM SUY DIEN CO GIAI THICH

def forward_chaining_co_giai_thich(su_kien, luat):
    facts = {f: "quan sát" for f in su_kien}

    thay_doi = True

    while thay_doi:
        thay_doi = False

        for r in luat:
            if r["thi"] not in facts and all(dk in facts for dk in r["neu"]):
```

```
            facts[r["thi"]] = (
                f"{r['ten']}: NEU {' VA '.join(r['neu'])} "
                f"THI {r['thi']}"
            )
            thay_doi = True

    return facts


# 3. HAM CHAN DOAN

def chan_doan(ten_ca, su_kien):
    facts = forward_chaining_co_giai_thich(su_kien, luat_mang)

    suy_ra = {
        k: v for k, v in facts.items()
        if v != "quan sát"
    }

    print("\n" + "=" * 65)
    print(ten_ca)
    print("=" * 65)

    print("Quan sát:", su_kien)

    if not suy_ra:
        print("-> Chưa đủ dữ kiện để kết luận.")
        return

    print("Kết quả suy luận:")
```

```
    for k, ly_do in suy_ra.items():
        print(" *", k, "<=", ly_do)


# 4. KIEM TRA 4 CA CU

chan_doan(
    "CA 1: LOI DNS",
    ["co_ip_hop_le", "ping_gateway_ok",
     "ping_8_8_8_8_ok", "phan_giai_ten_mien_that_bai"]
)

chan_doan(
    "CA 2: KHONG NHAN DUOC DHCP",
    ["ip_dang_169_254", "day_cap_ok"]
)

chan_doan(
    "CA 3: LOI KET NOI INTERNET",
    ["co_ip_hop_le", "ping_gateway_ok",
     "ping_8_8_8_8_that_bai"]
)

chan_doan(
    "CA 4: CHI CO IP HOP LE",
    ["co_ip_hop_le"]
)
```

```
# 5. CA MOI: KIEM TRA LOI PROXY / TRINH DUYET

chan_doan(
    "CA 5: PING INTERNET VA DNS OK NHUNG WEB KHONG MO",
    ["ping_8_8_8_8_ok",
     "phan_giai_ten_mien_ok",
     "web_khong_mo"]
)



# 6. CA MOI: KIEM TRA WIFI VA XUNG DOT IP

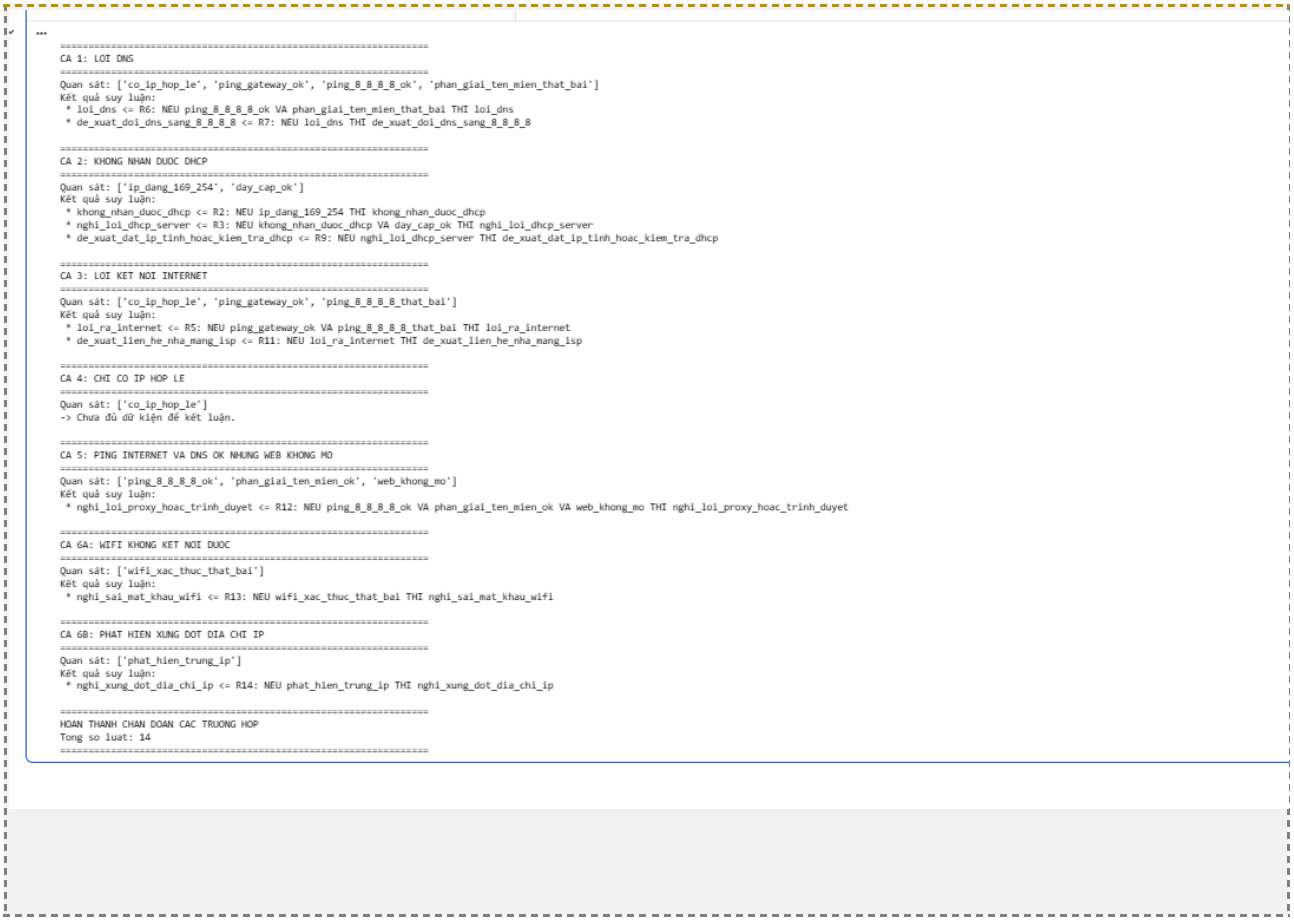
chan_doan(
    "CA 6A: WIFI KHONG KET NOI DUOC",
    ["wifi_xac_thuc_that_bai"]
)

chan_doan(
    "CA 6B: PHAT HIEN XUNG DOT DIA CHI IP",
    ["phat_hien_trung_ip"]
)



# 7. KET THUC

print("\n" + "=" * 65)
print("HOAN THANH CHAN DOAN CAC TRUONG HOP")
print("Tong so luat:", len(luat_mang))
print("=" * 65)
```



```
============================================================
CA 1: LOI DNS
============================================================
Quan sát: ['co_ip_hop_le', 'ping_gateway_ok', 'ping_8_8_8_8_ok', 'phan_giai_ten_mien_that_bai']
Kết quả suy luận:
 * loi_dns <= R6: NEU ping_8_8_8_8_ok VA phan_giai_ten_mien_that_bai THI loi_dns
 * de_xuat_doi_dns_sang_8_8_8_8 <= R7: NEU loi_dns THI de_xuat_doi_dns_sang_8_8_8_8

============================================================
CA 2: KHONG NHAN DUOC DHCP
============================================================
Quan sát: ['ip_dang_169_254', 'day_cap_ok']
Kết quả suy luận:
 * khong_nhan_duoc_dhcp <= R2: NEU ip_dang_169_254 THI khong_nhan_duoc_dhcp
 * nghi_loi_dhcp_server <= R3: NEU khong_nhan_duoc_dhcp VA day_cap_ok THI nghi_loi_dhcp_server
 * de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp <= R9: NEU nghi_loi_dhcp_server THI de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp

============================================================
CA 3: LOI KET NOI INTERNET
============================================================
Quan sát: ['co_ip_hop_le', 'ping_gateway_ok', 'ping_8_8_8_8_that_bai']
Kết quả suy luận:
 * loi_ra_internet <= R5: NEU ping_gateway_ok VA ping_8_8_8_8_that_bai THI loi_ra_internet
 * de_xuat_lien_he_nha_mang_isp <= R11: NEU loi_ra_internet THI de_xuat_lien_he_nha_mang_isp

============================================================
CA 4: CHI CO IP HOP LE
============================================================
Quan sát: ['co_ip_hop_le']
-> Chưa đủ dữ kiện để kết luận.

============================================================
CA 5: PING INTERNET VA DNS OK NHUNG WEB KHONG MO
============================================================
Quan sát: ['ping_8_8_8_8_ok', 'phan_giai_ten_mien_ok', 'web_khong_mo']
Kết quả suy luận:
 * nghi_loi_proxy_hoac_trinh_duyet <= R12: NEU ping_8_8_8_8_ok VA phan_giai_ten_mien_ok VA web_khong_mo THI nghi_loi_proxy_hoac_trinh_duyet

============================================================
CA 6A: WIFI KHONG KET NOI DUOC
============================================================
Quan sát: ['wifi_xac_thuc_that_bai']
Kết quả suy luận:
 * nghi_sai_mat_khau_wifi <= R13: NEU wifi_xac_thuc_that_bai THI nghi_sai_mat_khau_wifi

============================================================
CA 6B: PHAT HIEN XUNG DOT DIA CHI IP
============================================================
Quan sát: ['phat_hien_trung_ip']
Kết quả suy luận:
 * nghi_xung_dot_dia_chi_ip <= R14: NEU phat_hien_trung_ip THI nghi_xung_dot_dia_chi_ip

============================================================
HOAN THANH CHAN DOAN CAC TRUONG HOP
Tong so luat: 14
============================================================
```

Bài 2 — Suy diễn lùi (Backward Chaining): chỉ hỏi những gì cần thiết

Liên hệ CNTT: suy diễn tiến đi từ dữ kiện đến kết luận; suy diễn lùi đi từ GIẢ THUYẾT cần chứng minh ngược về dữ kiện — đây là cách Prolog hoạt động và cũng là cách các chatbot hỗ trợ kỹ thuật chọn câu hỏi tiếp theo. Cùng bộ luật mạng của Bài 1.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
luat_mang = [
    {"ten": "R1",  "neu": ["den_link_tat"], "thi": "loi_lop_vat_ly"},
    {"ten": "R2",  "neu": ["ip_dang_169_254"], "thi": "khong_nhan_duoc_dhcp"},
              {"ten":   "R3",       "neu":   ["khong_nhan_duoc_dhcp",   "day_cap_ok"],   "thi":
"nghi_loi_dhcp_server"},
    {"ten": "R4",  "neu": ["co_ip_hop_le", "ping_gateway_that_bai"], "thi": "loi_mang_noi_bo"},
            {"ten":   "R5",      "neu":   ["ping_gateway_ok",   "ping_8_8_8_8_that_bai"],   "thi":
"loi_ra_internet"},
    {"ten": "R6",  "neu": ["ping_8_8_8_8_ok", "phan_giai_ten_mien_that_bai"], "thi": "loi_dns"},
    {"ten": "R7",  "neu": ["loi_dns"], "thi": "de_xuat_doi_dns_sang_8_8_8_8"},
    {"ten": "R8",  "neu": ["loi_lop_vat_ly"], "thi": "de_xuat_kiem_tra_day_cap_va_card_mang"},
                      {"ten":     "R9",          "neu":     ["nghi_loi_dhcp_server"],     "thi":
"de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp"},
    {"ten": "R10", "neu": ["loi_mang_noi_bo"], "thi": "de_xuat_kiem_tra_switch_va_gateway"},
    {"ten": "R11", "neu": ["loi_ra_internet"], "thi": "de_xuat_lien_he_nha_mang_isp"},
]

def suy_dien_lui(muc_tieu, luat, hoi, da_biet, dem):
    """Chứng minh muc_tieu bằng cách đi ngược. Nếu muc_tieu không phải kết luận của luật nào ->
hỏi người dùng."""
    if muc_tieu in da_biet:
        return da_biet[muc_tieu]
    luat_ket_luan = [r for r in luat if r["thi"] == muc_tieu]
    if not luat_ket_luan:                              # sự kiện gốc: chỉ có thể hỏi
        da_biet[muc_tieu] = hoi(muc_tieu)
        dem["cau_hoi"] += 1
        return da_biet[muc_tieu]
    for r in luat_ket_luan:
        dem["luat_da_xet"] += 1
        if all(suy_dien_lui(dk, luat, hoi, da_biet, dem) for dk in r["neu"]):
```

```
            da_biet[muc_tieu] = True
            return True
    da_biet[muc_tieu] = False
    return False

# Người dùng "giả lập": chúng ta biết trước máy này gặp sự cố DNS
su_that = {"co_ip_hop_le": True, "ping_gateway_ok": True, "ping_8_8_8_8_ok": True,
           "phan_giai_ten_mien_that_bai": True}

def hoi(su_kien):
    tra_loi = su_that.get(su_kien, False)
    print(f"   ? {su_kien} -> {'Có' if tra_loi else 'Không'}")
    return tra_loi

so_su_kien_goc = len({dk for r in luat_mang for dk in r["neu"]} - {r["thi"] for r in luat_mang})
print("Tổng số sự kiện gốc mà suy diễn tiến cần biết trước:", so_su_kien_goc, "\n")

for muc_tieu in ["loi_dns", "loi_ra_internet", "nghi_loi_dhcp_server"]:
    print(f"Giả thuyết cần kiểm tra: {muc_tieu}")
    dem = {"cau_hoi": 0, "luat_da_xet": 0}
    ket_qua = suy_dien_lui(muc_tieu, luat_mang, hoi, {}, dem)
    print(f"   => {'ĐÚNG' if ket_qua else 'SAI'} | số câu hỏi đã hỏi: {dem['cau_hoi']} | số luật
đã xét: {dem['luat_da_xet']}\n")
```

```
            print(f"   => {'ĐÚNG' if ket_qua else 'SAI'} | số câu hỏi đã hỏi: {dem['cau_hoi']} | số
```

```
Tổng số sự kiện gốc mà suy diễn tiến cần biết trước: 9

Giả thuyết cần kiểm tra: loi_dns
   ? ping_8_8_8_8_ok -> Có
   ? phan_giai_ten_mien_that_bai -> Có
   => ĐÚNG | số câu hỏi đã hỏi: 2 | số luật đã xét: 1

Giả thuyết cần kiểm tra: loi_ra_internet
   ? ping_gateway_ok -> Có
   ? ping_8_8_8_8_that_bai -> Không
   => SAI | số câu hỏi đã hỏi: 2 | số luật đã xét: 1

Giả thuyết cần kiểm tra: nghi_loi_dhcp_server
   ? ip_dang_169_254 -> Không
   => SAI | số câu hỏi đã hỏi: 1 | số luật đã xét: 2
```

💭 **Suy nghĩ: Với giả thuyết loi_dns, suy diễn lùi chỉ cần hỏi bao nhiêu câu so với tổng số sự kiện gốc của hệ thống? Vì sao với giả thuyết nghi_loi_dhcp_server (kết quả SAI) hệ thống chỉ hỏi đúng 1 câu rồi dừng? Khi nào nên chọn suy diễn tiến, khi nào nên chọn suy diễn lùi?**

1. Với giả thuyết `loi_dns`, suy diễn lùi chỉ cần hỏi bao nhiêu câu?

Hệ thống chỉ cần hỏi 2 câu, so với tổng cộng 9 sự kiện gốc mà hệ thống suy diễn tiến cần biết trước.

Hai câu hỏi là:

- `ping_8_8_8_8_ok` → Có.
- `phan_giai_ten_mien_that_bai` → Có.

Vì cả hai điều kiện của luật R6 đều thỏa mãn nên hệ thống kết luận `loi_dns` là đúng.

2. Vì sao giả thuyết `nghi_loi_dhcp_server` chỉ hỏi 1 câu rồi dừng?

Vì luật R3 yêu cầu đồng thời:

- `khong_nhan_duoc_dhcp`

- `day_cap_ok`

Hệ thống kiểm tra điều kiện đầu tiên bằng cách hỏi `ip_dang_169_254` và nhận câu trả lời Không. Do đó, điều kiện của R2 không thỏa mãn, khiến R3 không thể được kích hoạt.

Vì sử dụng logic AND, chỉ cần một điều kiện sai thì hệ thống không cần kiểm tra tiếp các điều kiện còn lại.

3. Khi nào nên chọn suy diễn tiến, khi nào nên chọn suy diễn lùi?

Tiêu chí	Suy diễn tiến	Suy diễn lùi
Bắt đầu từ	Dữ kiện ban đầu	Giả thuyết cần kiểm tra
Hướng suy luận	Dữ kiện ⟶ Kết luận	Kết luận ⟶ Dữ kiện
Phù hợp khi	Muốn tìm tất cả kết luận có thể	Muốn kiểm tra một kết luận cụ thể
Ví dụ	Giám sát và phát hiện lỗi mạng	Kiểm tra máy có bị lỗi DNS không

Ví dụ thực tế:

- Suy diễn tiến: Hệ thống giám sát mạng liên tục thu thập dữ liệu và tự động phát hiện các lỗi có thể xảy ra.
- Suy diễn lùi: Người dùng nghi ngờ máy bị lỗi DNS, hệ thống đặt giả thuyết này rồi hỏi các câu cần thiết để xác minh.

Kết luận

Suy diễn tiến phù hợp khi có nhiều dữ kiện và muốn tìm ra các kết luận có thể xảy ra. Suy diễn lùi phù hợp khi đã có một giả thuyết cụ thể và muốn kiểm tra xem giả thuyết đó có đúng hay không.

Điểm nổi bật: Suy diễn lùi có thể tiết kiệm số câu hỏi và dữ kiện cần thu thập vì chỉ tập trung vào những điều kiện liên quan đến giả thuyết đang kiểm tra.

🔧 Cải tiến: Thêm giả thuyết de_xuat_doi_dns_sang_8_8_8_8 (một kết luận nằm ở tầng sâu hơn) và so sánh số câu hỏi. Sau đó thay hàm hoi bằng phiên bản tương tác thật: def hoi(sk): return input(f"{sk}? (c/k) ").strip().lower() == "c" và tự chơi thử như một người dùng.

```
# ========================================
```

```
# HE CHUYEN GIA CHAN DOAN LOI MANG
# SUY DIEN LUI - BACKWARD CHAINING
# ========================================


# 1. CO SO TRI THUC: 11 LUAT IF-THEN

luat_mang = [
    {"ten": "R1", "neu": ["den_link_tat"], "thi": "loi_lop_vat_ly"},
    {"ten": "R2", "neu": ["ip_dang_169_254"], "thi": "khong_nhan_duoc_dhcp"},
    {"ten": "R3", "neu": ["khong_nhan_duoc_dhcp", "day_cap_ok"], "thi": "nghi_loi_dhcp_server"},
    {"ten": "R4", "neu": ["co_ip_hop_le", "ping_gateway_that_bai"], "thi": "loi_mang_noi_bo"},
    {"ten": "R5", "neu": ["ping_gateway_ok", "ping_8_8_8_8_that_bai"], "thi": "loi_ra_internet"},
    {"ten": "R6", "neu": ["ping_8_8_8_8_ok", "phan_giai_ten_mien_that_bai"], "thi": "loi_dns"},
    {"ten": "R7", "neu": ["loi_dns"], "thi": "de_xuat_doi_dns_sang_8_8_8_8"},
    {"ten": "R8", "neu": ["loi_lop_vat_ly"], "thi": "de_xuat_kiem_tra_day_cap_va_card_mang"},
    {"ten": "R9", "neu": ["nghi_loi_dhcp_server"], "thi": "de_xuat_dat_ip_tinh_hoac_kiem_tra_dhcp"},
    {"ten": "R10", "neu": ["loi_mang_noi_bo"], "thi": "de_xuat_kiem_tra_switch_va_gateway"},
    {"ten": "R11", "neu": ["loi_ra_internet"], "thi": "de_xuat_lien_he_nha_mang_isp"},
]



# 2. HAM HOI NGUOI DUNG TUONG TAC

def hoi(sk):
    return input(f"{sk}? (c/k) ").strip().lower() == "c"



# 3. HAM SUY DIEN LUI

def suy_dien_lui(muc_tieu, luat, hoi, da_biet, dem):
```

```
# Neu da biet su kien thi tra ve ket qua
if muc_tieu in da_biet:
    return da_biet[muc_tieu]

# Tim cac luat co ket luan trung voi muc tieu
luat_ket_luan = [
    r for r in luat if r["thi"] == muc_tieu
]

# Neu khong co luat nao thi hoi nguoi dung
if not luat_ket_luan:
    da_biet[muc_tieu] = hoi(muc_tieu)
    dem["cau_hoi"] += 1
    return da_biet[muc_tieu]

# Kiem tra cac dieu kien cua luat
for r in luat_ket_luan:
    dem["luat_da_xet"] += 1

    if all(
        suy_dien_lui(dk, luat, hoi, da_biet, dem)
        for dk in r["neu"]
    ):
        da_biet[muc_tieu] = True
        return True

da_biet[muc_tieu] = False
return False
```

```
# 4. DEM SO SU KIEN GOC

so_su_kien_goc = len(
    {dk for r in luat_mang for dk in r["neu"]}
    - {r["thi"] for r in luat_mang}
)

print("=" * 60)
print("HE CHUYEN GIA CHAN DOAN LOI MANG")
print("Tong so luat:", len(luat_mang))
print("Tong su kien goc:", so_su_kien_goc)
print("=" * 60)

print("\nHUONG DAN:")
print("Nhap c neu cau tra loi la CO")
print("Nhap k neu cau tra loi la KHONG")


# 5. THU NGHIEM 3 GIA THUYET

cac_muc_tieu = [
    "loi_dns",
    "de_xuat_doi_dns_sang_8_8_8_8",
    "nghi_loi_dhcp_server"
]

ket_qua_thu_nghiem = []

for muc_tieu in cac_muc_tieu:
```

```
print("\n" + "-" * 60)
print("GIA THUYET CAN KIEM TRA:", muc_tieu)

dem = {
    "cau_hoi": 0,
    "luat_da_xet": 0
}

# Moi gia thuyet duoc kiem tra doc lap
da_biet = {}

ket_qua = suy_dien_lui(
    muc_tieu,
    luat_mang,
    hoi,
    da_biet,
    dem
)

print(
    "=>",
    "DUNG" if ket_qua else "SAI",
    "| So cau hoi:", dem["cau_hoi"],
    "| So luat da xet:", dem["luat_da_xet"]
)

ket_qua_thu_nghiem.append({
    "muc_tieu": muc_tieu,
    "ket_qua": ket_qua,
```

```
        "cau_hoi": dem["cau_hoi"],
        "luat": dem["luat_da_xet"]
    })


# 6. SO SANH KET QUA

print("\n" + "=" * 60)
print("BANG SO SANH KET QUA")
print("=" * 60)

for kq in ket_qua_thu_nghiem:
    print("\nGia thuyet:", kq["muc_tieu"])
    print("Ket qua:", "DUNG" if kq["ket_qua"] else "SAI")
    print("So cau hoi:", kq["cau_hoi"])
    print("So luat da xet:", kq["luat"])

print("\nHOAN THANH CHUONG TRINH!")
```

```
============================================================
HE CHUYEN GIA CHAN DOAN LOI MANG
Tong so luat: 11
Tong su kien goc: 9
============================================================

HUONG DAN:
Nhap c neu cau tra loi la CO
Nhap k neu cau tra loi la KHONG

------------------------------------------------------------
GIA THUYET CAN KIEM TRA: loi_dns
ping_8_8_8_8_ok? (c/k) c
phan_giai_ten_mien_that_bai? (c/k) c
=> DUNG | So cau hoi: 2 | So luat da xet: 1

------------------------------------------------------------
GIA THUYET CAN KIEM TRA: de_xuat_doi_dns_sang_8_8_8_8
ping_8_8_8_8_ok? (c/k) c
phan_giai_ten_mien_that_bai? (c/k) c
=> DUNG | So cau hoi: 2 | So luat da xet: 2

------------------------------------------------------------
GIA THUYET CAN KIEM TRA: nghi_loi_dhcp_server
ip_dang_169_254? (c/k) c
day_cap_ok? (c/k) c
=> DUNG | So cau hoi: 2 | So luat da xet: 2

============================================================
BANG SO SANH KET QUA
============================================================

Gia thuyet: loi_dns
Ket qua: DUNG
So cau hoi: 2
So luat da xet: 1

Gia thuyet: de_xuat_doi_dns_sang_8_8_8_8
Ket qua: DUNG
So cau hoi: 2
So luat da xet: 2

Gia thuyet: nghi_loi_dhcp_server
Ket qua: DUNG
So cau hoi: 2
So luat da xet: 2

HOAN THANH CHUONG TRINH!
```

Bài 3 — Kiểm định bộ luật phân quyền: phát hiện luật thừa, mâu thuẫn và luật "chết"

Liên hệ CNTT: chính sách phân quyền, tường lửa (firewall), IAM đều là bộ luật NẾU–THÌ. Bộ luật lớn dần thì sinh ra luật thừa, luật mâu thuẫn (một luật cho phép, một luật chặn) và luật không bao giờ được kích hoạt — đây là nguồn gốc của nhiều lỗ hổng bảo mật. Việc kiểm tra tĩnh bộ luật là kỹ năng quan trọng khi làm DevSecOps.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
luat_chinh_sach = [
    {"ten": "P1", "neu": ["la_nhan_vien"], "thi": "cho_phep_vpn"},
    {"ten": "P2", "neu": ["la_nhan_vien", "thiet_bi_cong_ty"], "thi": "cho_phep_vpn"},
    {"ten": "P3", "neu": ["la_khach"], "thi": "chan_vpn"},
    {"ten": "P4", "neu": ["la_nhan_vien", "dang_nghi_viec"], "thi": "chan_vpn"},
                {"ten":    "P5",    "neu":    ["la_quan_tri",    "da_xac_thuc_2_buoc"],    "thi":
"cho_phep_truy_cap_server"},
           {"ten":   "P6",   "neu":   ["cho_phep_truy_cap_server",   "trong_gio_lam_viec"],   "thi":
"cho_phep_deploy"},
    {"ten": "P7", "neu": ["la_quan_tri", "chua_xac_thuc"], "thi": "chan_truy_cap_server"},
              {"ten":   "P8",   "neu":   ["la_thuc_tap_sinh",   "co_giay_phep_dac_biet"],   "thi":
"cho_phep_deploy"},
    {"ten": "P9", "neu": ["la_quan_tri", "la_khach"], "thi": "cho_phep_vpn"},
]

# Các sự kiện hệ thống thực sự quan sát được (đầu vào của hệ chuyên gia)
su_kien_quan_sat    =    {"la_nhan_vien",    "la_khach",    "la_quan_tri",    "la_thuc_tap_sinh",
"thiet_bi_cong_ty",
                                      "dang_nghi_viec",  "da_xac_thuc_2_buoc",  "chua_xac_thuc",
"trong_gio_lam_viec"}
```

```
# Các cặp sự kiện không thể cùng đúng
loai_tru = [("la_nhan_vien", "la_khach"), ("la_quan_tri", "la_khach"), ("da_xac_thuc_2_buoc",
"chua_xac_thuc")]

def co_loai_tru(tap_su_kien):
    return any(a in tap_su_kien and b in tap_su_kien for a, b in loai_tru)

def doi_lap(kl1, kl2):
    return (kl1.startswith("cho_phep_") and kl2 == "chan_" + kl1[len("cho_phep_"):]) or \
           (kl2.startswith("cho_phep_") and kl1 == "chan_" + kl2[len("cho_phep_"):])

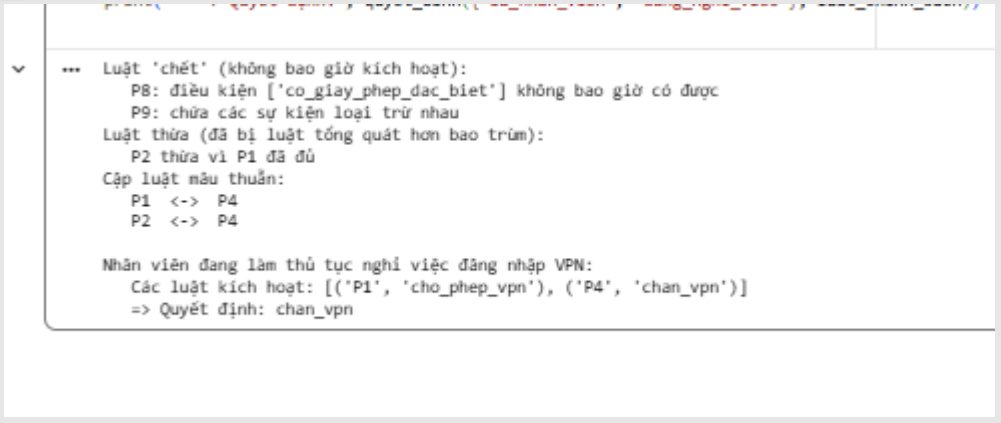
def kiem_dinh(luat):
    co_the_suy_ra = su_kien_quan_sat | {r["thi"] for r in luat}
    # 1) Luật "chết": chứa sự kiện mâu thuẫn nhau, hoặc chứa sự kiện không bao giờ có được
    chet = []
    for r in luat:
        if co_loai_tru(set(r["neu"])):
            chet.append((r["ten"], "chứa các sự kiện loại trừ nhau"))
        elif any(dk not in co_the_suy_ra for dk in r["neu"]):
            thieu = [dk for dk in r["neu"] if dk not in co_the_suy_ra]
            chet.append((r["ten"], f"điều kiện {thieu} không bao giờ có được"))
    ten_chet = {t for t, _ in chet}
    song = [r for r in luat if r["ten"] not in ten_chet]
    # 2) Luật thừa: cùng kết luận, và có luật khác tổng quát hơn (điều kiện là tập con)
    thua = [(a["ten"], b["ten"]) for a in song for b in song
            if a is not b and a["thi"] == b["thi"] and set(b["neu"]) < set(a["neu"])]
    # 3) Mâu thuẫn: hai luật kết luận đối lập nhau mà điều kiện có thể cùng đúng
    mau_thuan = [(a["ten"], b["ten"]) for i, a in enumerate(song) for b in song[i + 1:]
                 if doi_lap(a["thi"], b["thi"]) and not co_loai_tru(set(a["neu"]) |
set(b["neu"]))]
    return chet, thua, mau_thuan

chet, thua, mau_thuan = kiem_dinh(luat_chinh_sach)
print("Luật 'chết' (không bao giờ kích hoạt):")
for t, ly_do in chet: print(f"   {t}: {ly_do}")
print("Luật thừa (đã bị luật tổng quát hơn bao trùm):")
for a, b in thua: print(f"   {a} thừa vì {b} đã đủ")
print("Cặp luật mâu thuẫn:")
for a, b in mau_thuan: print(f"   {a}  <->  {b}")

# Giải quyết xung đột bằng "đặc thù hơn thắng": luật có nhiều điều kiện hơn được ưu tiên
def quyet_dinh(su_kien, luat):
    kich_hoat = [r for r in luat if all(dk in su_kien for dk in r["neu"]) and not
co_loai_tru(set(r["neu"]))]
    print("   Các luật kích hoạt:", [(r["ten"], r["thi"]) for r in kich_hoat])
    return max(kich_hoat, key=lambda r: len(r["neu"]))["thi"]

print("\nNhân viên đang làm thủ tục nghỉ việc đăng nhập VPN:")
print("   => Quyết định:", quyet_dinh({"la_nhan_vien", "dang_nghi_viec"}, luat_chinh_sach))
```

📷 ***Dán ảnh kết quả kiểm định bộ luật và quyết định cuối cùng***



💭 **Suy nghĩ: Vì sao P9 và P8 là luật "chết" nhưng lý do lại khác nhau? Vì sao P1 và P3 không bị coi là mâu thuẫn dù một luật cho phép, một luật chặn? Chiến lược "luật đặc thù hơn thắng" có thể sai trong trường hợp nào (gợi ý: nghĩ đến nguyên tắc "từ chối mặc định" trong bảo mật)?**

1. Vì sao P8 và P9 đều là luật chết nhưng lý do khác nhau?

- P8 – Điều kiện không bao giờ có được: P8 yêu cầu `co_giay_phep_dac_biet`, nhưng sự kiện này không nằm trong tập sự kiện quan sát và cũng không được luật nào khác suy ra. Vì vậy, P8 không thể kích hoạt.
- P9 – Chứa sự kiện loại trừ nhau: P9 yêu cầu đồng thời `la_quan_tri` và `la_khach`. Tuy nhiên, hai sự kiện này được quy định là không thể cùng đúng. Vì vậy, P9 không thể kích hoạt.

2. Vì sao P1 và P3 không bị coi là mâu thuẫn?

- P1: `la_nhan_vien` → `cho_phep_vpn`.
- P3: `la_khach` → `chan_vpn`.

Mặc dù hai luật đưa ra kết luận đối lập, nhưng điều kiện của chúng là `la_nhan_vien` và `la_khach` – hai sự kiện loại trừ nhau.

Do đó, không thể có trường hợp cả P1 và P3 cùng kích hoạt trên một đối tượng. Vì vậy, chúng không bị xem là mâu thuẫn.

3. Khi nào chiến lược "luật đặc thù hơn thắng" có thể sai?

Chiến lược này có thể sai khi luật đặc thù hơn cho phép truy cập nhưng bỏ qua các nguyên tắc bảo mật quan trọng.

Ví dụ:

- P1: Nhân viên được phép sử dụng VPN.
- P4: Nhân viên đang nghỉ việc bị chặn VPN.

Nếu có thêm một luật đặc thù hơn cho phép nhân viên nghỉ việc truy cập VPN, hệ thống có thể ưu tiên luật đó chỉ vì nó có nhiều điều kiện hơn, dẫn đến cấp quyền trái chính sách.

Trong bảo mật, cần áp dụng nguyên tắc Từ chối mặc định (Default Deny): nếu không có quy tắc rõ ràng cho phép truy cập thì phải từ chối.

Kết luận: Không nên chỉ dựa vào số lượng điều kiện để quyết định luật thắng. Cần ưu tiên các quy tắc bảo mật quan trọng như thu hồi quyền, trạng thái tài khoản và xác thực; khi có xung đột chưa được giải quyết rõ ràng, nên từ chối truy cập để đảm bảo an toàn.

🔧 Cải tiến: Thêm 2 luật mới: một luật cố tình mâu thuẫn với luật có sẵn, một luật cố tình thừa. Chạy lại kiem_dinh để xem nó có phát hiện đúng không. Sau đó đổi quyet_dinh sang chiến lược "chặn thắng" (nếu có bất kỳ luật chặn nào kích hoạt thì chặn) và so sánh kết quả.

```
# P10: Luật cố tình tạo mâu thuẫn với P1

# P11: Luật cố tình thừa so với P1



luat_chinh_sach.extend([

  {
```

```
    "ten": "P10",
    "neu": ["la_nhan_vien", "thiet_bi_ca_nhan"],
    "thi": "chan_vpn"
  },
  {
    "ten": "P11",
    "neu": ["la_nhan_vien", "thiet_bi_cong_ty"],
    "thi": "cho_phep_vpn"
  }
])

# Bổ sung sự kiện thiết bị cá nhân vào tập sự kiện quan sát
su_kien_quan_sat.add("thiet_bi_ca_nhan")

chet, thua, mau_thuan = kiem_dinh(luat_chinh_sach)

print("Luật chết:")
for t, ly_do in chet:
  print(f"{t}: {ly_do}")

print("\nLuật thừa:")
for a, b in thua:
  print(f"{a} thừa vì {b} đã đủ")

print("\nCặp luật mâu thuẫn:")
for a, b in mau_thuan:
  print(f"{a} <-> {b}")
```

```
    print("\nCặp luật mâu thuẫn:")
    for a, b in mau_thuan:
        print(f"{a} <-> {b}")
```

```
Luật chết:
P8: điều kiện ['co_giay_phep_dac_biet'] không bao giờ có được
P9: chứa các sự kiện loại trừ nhau

Luật thừa:
P2 thừa vì P1 đã đủ
P11 thừa vì P1 đã đủ

Cặp luật mâu thuẫn:
P1 <-> P4
P1 <-> P10
P2 <-> P4
P2 <-> P10
P4 <-> P11
P10 <-> P11
```

Bài 4 — Luật có độ tin cậy (Certainty Factor): bộ lọc thư rác / lừa đảo

Liên hệ CNTT: bộ lọc thư rác đời đầu (như SpamAssassin) cộng điểm từ rất nhiều luật yếu thay vì kết luận tuyệt đối. Mô hình hệ số tin cậy (CF) của hệ chuyên gia MYCIN cho phép nhiều bằng chứng yếu cùng tăng niềm tin, và bằng chứng trái chiều làm giảm niềm tin.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
luat_cf = [
    {"ten": "S1", "neu": ["tieu_de_viet_hoa"],                          "thi": "spam",     "cf":
0.4},
    {"ten": "S2", "neu": ["chua_link_rut_gon"],                         "thi": "spam",     "cf":
0.5},
    {"ten": "S3", "neu": ["nguoi_gui_la", "yeu_cau_gui_otp"],          "thi": "phishing", "cf":
0.9},
    {"ten": "S4", "neu": ["tep_dinh_kem_exe"],                         "thi": "phishing", "cf":
0.8},
    {"ten": "S5", "neu": ["nguoi_gui_trong_danh_ba"],                  "thi": "spam",     "cf":
-0.7},
    {"ten": "S6", "neu": ["phishing"],                                 "thi": "spam",     "cf":
0.95},
    {"ten": "S7", "neu": ["noi_dung_gian_gap", "yeu_cau_chuyen_tien"], "thi": "phishing", "cf":
0.7},
]

def ket_hop_hai(a, b):
    if a >= 0 and b >= 0: return a + b * (1 - a)
    if a <= 0 and b <= 0: return a + b * (1 + a)
    return (a + b) / (1 - min(abs(a), abs(b)))

def ket_hop_nhieu(ds):
    duong = [x for x in ds if x > 0]
    am = [x for x in ds if x < 0]
    tong_duong = 0.0
    for x in duong: tong_duong = ket_hop_hai(tong_duong, x)
    tong_am = 0.0
    for x in am: tong_am = ket_hop_hai(tong_am, x)
    return ket_hop_hai(tong_duong, tong_am)

def suy_dien_cf(su_kien, luat):
    """su_kien: {tên_sự_kiện: độ_chắc_chắn từ 0 đến 1}. Trả về CF của các kết luận."""
    cf = dict(su_kien)
    for _ in range(10):                                  # lặp đến khi ổn định
        gom = {}
        for r in luat:
            if all(dk in cf for dk in r["neu"]):
                do_chac = min(cf[dk] for dk in r["neu"])  # điều kiện AND -> lấy độ chắc chắn
thấp nhất
                gom.setdefault(r["thi"], []).append(r["cf"] * do_chac)
        moi = {k: ket_hop_nhieu(ds) for k, ds in gom.items()}
        if all(abs(cf.get(k, 99) - v) < 1e-9 for k, v in moi.items()):
            break
        cf.update(moi)
    return {k: v for k, v in cf.items() if k in {r["thi"] for r in luat}}
```

```
def phan_loai(diem):
    if diem >= 0.8: return "CHẶN"
    if diem >= 0.5: return "ĐƯA VÀO THƯ RÁC"
    return "CHO PHÉP"

thu = {
     "Thư 1: tiêu đề VIẾT HOA + link rút gọn": {"tieu_de_viet_hoa": 1.0, "chua_link_rut_gon": 1.0},
     "Thư 2: người gửi lạ xin OTP + link rút gọn": {"nguoi_gui_la": 1.0, "yeu_cau_gui_otp": 0.8, "chua_link_rut_gon": 1.0},
       "Thư 3: giống thư 1 nhưng gửi từ người trong danh bạ": {"tieu_de_viet_hoa": 1.0, "chua_link_rut_gon": 1.0, "nguoi_gui_trong_danh_ba": 1.0},
}
for ten, su_kien in thu.items():
    kq = suy_dien_cf(su_kien, luat_cf)
    spam = kq.get("spam", 0.0)
     print(f"{ten}\n   CF: " + ", ".join(f"{k}={v:.3f}" for k, v in sorted(kq.items())) + f"  => {phan_loai(spam)}\n")
```

```
spam = kq.get("spam", 0.0)
print(f"{ten}\n   CF: " + ", ".join(f"{k}={v:.3f}" for k, v in sorted(kq.items())) + f"
```

```
Thư 1: tiêu đề VIẾT HOA + link rút gọn
   CF: spam=0.700  =>  ĐƯA VÀO THƯ RÁC

Thư 2: người gửi lạ xin OTP + link rút gọn
   CF: phishing=0.720, spam=0.842  =>  CHẶN

Thư 3: giống thư 1 nhưng gửi từ người trong danh bạ
   CF: spam=0.000  =>  CHO PHÉP
```

💭 **Suy nghĩ: Vì sao hai bằng chứng yếu (0,4 và 0,5) khi kết hợp cho 0,7 chứ không phải 0,9 hay 0,45? Ở Thư 3, bằng chứng trái chiều làm điều gì với kết luận spam? Việc dùng min cho điều kiện AND nghĩa là gì khi một dữ kiện (yeu_cau_gui_otp = 0,8) không chắc chắn?**

Vì sao hai bằng chứng yếu (0,4 và 0,5) kết hợp thành 0,7?

Hệ thống sử dụng công thức kết hợp hai bằng chứng cùng chiều:

$$CF = a + b(1 - a)$$

Thay $a = 0,4, b = 0,5$:

$$CF = 0,4 + 0,5(1 - 0,4) = 0,7$$

Kết quả là 0,7, không phải 0,9 vì hệ thống không cộng trực tiếp hai độ tin cậy. Cũng không phải 0,45 vì không lấy trung bình.

Ý nghĩa: Hai bằng chứng cùng ủng hộ kết luận spam sẽ làm độ tin cậy tăng lên, nhưng không vượt quá 1.

Ở Thư 3, bằng chứng trái chiều làm gì với kết luận spam?

Thư 3 có:

- Tiêu đề viết hoa: CF = +0,4.
- Có link rút gọn: CF = +0,5.
- Người gửi trong danh bạ: CF = -0,7.

Hai bằng chứng đầu ủng hộ spam, kết hợp thành 0,7. Bằng chứng người gửi trong danh bạ lại phản đối spam với CF = -0,7.

Kết hợp hai bằng chứng trái chiều:

$$CF = \frac{0{,}7+(-0{,}7)}{1-\min(0{,}7;0{,}7)} = 0$$

Vì vậy, độ tin cậy của kết luận spam giảm về 0 và hệ thống đưa ra kết quả CHO PHÉP theo ngưỡng phân loại đã thiết lập.

Dùng `min` cho điều kiện AND có ý nghĩa gì?

Trong luật S3, điều kiện gồm:

- Người gửi lạ: CF = 1,0.
- Yêu cầu gửi OTP: CF = 0,8.

Hệ thống lấy giá trị nhỏ nhất:

$$CF_{AND} = \min(1,0;0,8) = 0,8$$

Sau đó nhân với CF của luật S3:

$$CF_{phishing} = 0,9 \times 0,8 = 0,72$$

Ý nghĩa: Khi một luật cần nhiều điều kiện cùng đúng, độ chắc chắn của cả nhóm điều kiện bị giới hạn bởi điều kiện kém chắc chắn nhất.

Kết luận: Hệ thống CF cho phép kết hợp nhiều bằng chứng ủng hộ và phản đối để đưa ra kết luận linh hoạt hơn. Nhờ đó, hệ thống không chỉ dựa vào một dấu hiệu đơn lẻ mà còn cân nhắc mức độ chắc chắn của từng dữ kiện.

🔧 Cải tiến: Thêm 2 luật mới của bạn (ví dụ: tên miền người gửi giả mạo tên miền ngân hàng → phishing 0,85; email có chữ ký số hợp lệ → spam −0,6), tạo thêm 2 thư thử nghiệm, và thử đổi ngưỡng phan_loai. Tìm một thư "khó" mà hệ thống phân loại sai và giải thích vì sao.

```
# ================================================
# HE CHUYEN GIA PHAN LOAI EMAIL SPAM / PHISHING
# SU DUNG CERTAINTY FACTOR (CF)
# ================================================
```

```
# 1. KHAI BAO 9 LUAT CF

luat_cf = [

  {"ten": "S1", "neu": ["tieu_de_viet_hoa"],

   "thi": "spam", "cf": 0.4},


  {"ten": "S2", "neu": ["chua_link_rut_gon"],

   "thi": "spam", "cf": 0.5},


  {"ten": "S3", "neu": ["nguoi_gui_la", "yeu_cau_gui_otp"],

   "thi": "phishing", "cf": 0.9},


  {"ten": "S4", "neu": ["tep_dinh_kem_exe"],

   "thi": "phishing", "cf": 0.8},


  {"ten": "S5", "neu": ["nguoi_gui_trong_danh_ba"],

   "thi": "spam", "cf": -0.7},


  {"ten": "S6", "neu": ["phishing"],

   "thi": "spam", "cf": 0.95},


  {"ten": "S7", "neu": ["noi_dung_gian_gap", "yeu_cau_chuyen_tien"],

   "thi": "phishing", "cf": 0.7},


  # Luat moi S8

  {"ten": "S8", "neu": ["ten_mien_gia_mao_ten_mien_ngan_hang"],

   "thi": "phishing", "cf": 0.85},


  # Luat moi S9

  {"ten": "S9", "neu": ["email_co_chu_ky_so_hop_le"],

   "thi": "spam", "cf": -0.6},
```

```
]



# 2. HAM KET HOP HAI GIA TRI CF

def ket_hop_hai(a, b):

    if a >= 0 and b >= 0:

        return a + b * (1 - a)


    if a <= 0 and b <= 0:

        return a + b * (1 + a)


    return (a + b) / (1 - min(abs(a), abs(b)))




# 3. HAM KET HOP NHIEU GIA TRI CF

def ket_hop_nhieu(ds):

    duong = [x for x in ds if x > 0]

    am = [x for x in ds if x < 0]


    tong_duong = 0.0

    for x in duong:

        tong_duong = ket_hop_hai(tong_duong, x)


    tong_am = 0.0

    for x in am:

        tong_am = ket_hop_hai(tong_am, x)


    return ket_hop_hai(tong_duong, tong_am)
```

```
# 4. HAM SUY DIEN CF
def suy_dien_cf(su_kien, luat):
    cf = dict(su_kien)

    for _ in range(10):
        gom = {}

        for r in luat:
            if all(dk in cf for dk in r["neu"]):
                do_chac = min(cf[dk] for dk in r["neu"])

                gom.setdefault(r["thi"], []).append(
                    r["cf"] * do_chac
                )

        moi = {
            k: ket_hop_nhieu(ds)
            for k, ds in gom.items()
        }

        if all(
            abs(cf.get(k, 99) - v) < 1e-9
            for k, v in moi.items()
        ):
            break

        cf.update(moi)

    return {
        k: v for k, v in cf.items()
```

```
        if k in {r["thi"] for r in luat}
    }



# 5. HAM PHAN LOAI CO THE THAY DOI NGUONG
def phan_loai(diem, nguong_chan=0.8, nguong_rac=0.5):
    if diem >= nguong_chan:
        return "CHẶN"

    if diem >= nguong_rac:
        return "ĐƯA VÀO THƯ RÁC"

    return "CHO PHÉP"



# 6. HAM PHAN LOAI NANG CAO
# Neu phishing >= 0.8 thi chan ngay
def phan_loai_nang_cao(kq, nguong_chan=0.8, nguong_rac=0.5):
    phishing = kq.get("phishing", 0.0)
    spam = kq.get("spam", 0.0)

    if phishing >= 0.8:
        return "CHẶN - NGHI PHISHING"

    return phan_loai(spam, nguong_chan, nguong_rac)



# 7. CAC EMAIL THU NGHIEM
thu = {
    "Thu 1: Tieu de viet hoa + link rut gon": {
```

```
    "tieu_de_viet_hoa": 1.0,
    "chua_link_rut_gon": 1.0
},

"Thu 2: Nguoi gui la xin OTP + link rut gon": {
    "nguoi_gui_la": 1.0,
    "yeu_cau_gui_otp": 0.8,
    "chua_link_rut_gon": 1.0
},

"Thu 3: Tieu de viet hoa + link rut gon, nguoi gui trong danh ba": {
    "tieu_de_viet_hoa": 1.0,
    "chua_link_rut_gon": 1.0,
    "nguoi_gui_trong_danh_ba": 1.0
},

# Email thu nghiem moi 4
"Thu 4: Gia mao ten mien ngan hang, xin OTP": {
    "ten_mien_gia_mao_ten_mien_ngan_hang": 1.0,
    "nguoi_gui_la": 1.0,
    "yeu_cau_gui_otp": 0.8
},

# Email thu nghiem moi 5
"Thu 5: Gia mao ngan hang nhung co chu ky so hop le": {
    "ten_mien_gia_mao_ten_mien_ngan_hang": 1.0,
    "nguoi_gui_la": 1.0,
    "yeu_cau_gui_otp": 0.8,
    "email_co_chu_ky_so_hop_le": 1.0
}
```

```
}



# 8. CHAY THU 5 EMAIL

print("=" * 65)

print("KET QUA PHAN LOAI 5 EMAIL")

print("=" * 65)


ket_qua = {}


for ten, su_kien in thu.items():

    kq = suy_dien_cf(su_kien, luat_cf)

    ket_qua[ten] = kq


    spam = kq.get("spam", 0.0)


    print(f"\n{ten}")

    print("CF:", ", ".join(

        f"{k}={v:.3f}" for k, v in sorted(kq.items())

    ))

    print("Phan loai:", phan_loai(spam))



# 9. SO SANH HAI BO NGUONG

print("\n" + "=" * 65)

print("SO SANH NGUONG PHAN LOAI")

print("=" * 65)


for ten, kq in ket_qua.items():

    spam = kq.get("spam", 0.0)
```

```
    cu = phan_loai(spam, 0.8, 0.5)
    moi = phan_loai(spam, 0.9, 0.6)

    print(f"\n{ten}")
    print(f"CF spam: {spam:.3f}")
    print(f"Nguong cu (0.8 / 0.5): {cu}")
    print(f"Nguong moi (0.9 / 0.6): {moi}")



# 10. PHAN LOAI NANG CAO
print("\n" + "=" * 65)
print("PHAN LOAI NANG CAO - KIEM TRA PHISHING RIENG")
print("=" * 65)

for ten, kq in ket_qua.items():
    print(f"\n{ten}")
    print("CF phishing:", round(kq.get("phishing", 0.0), 3))
    print("CF spam:", round(kq.get("spam", 0.0), 3))
    print("Ket luan:", phan_loai_nang_cao(kq))



# 11. TIM EMAIL CO NGUY CO PHAN LOAI SAI
print("\n" + "=" * 65)
print("PHAN TICH EMAIL KHO")
print("=" * 65)

ten_kho = "Thu 5: Gia mao ngan hang nhung co chu ky so hop le"
kq_kho = ket_qua[ten_kho]
```

```
print("Email:", ten_kho)
print("CF phishing:", round(kq_kho.get("phishing", 0.0), 3))
print("CF spam:", round(kq_kho.get("spam", 0.0), 3))

print("Phan loai theo spam:",
    phan_loai(kq_kho.get("spam", 0.0)))

print("Phan loai nang cao:",
    phan_loai_nang_cao(kq_kho))

print("\nNhan xet:")
print("- Chu ky so hop le khong dong nghia email an toan.")
print("- Bang chung am co the lam giam diem spam.")
print("- Neu chi kiem tra spam, he thong co the bo sot phishing.")
print("- Can kiem tra CF phishing rieng de tang kha nang phat hien.")
```

```
print("- Neu chi kiem tra spam, he thong co the bo sot phishing.")
print("- Can kiem tra CF phishing rieng de tang kha nang phat hien.")
```

```
...
```

```
================================================================
KET QUA PHAN LOAI 5 EMAIL
================================================================

Thu 1: Tieu de viet hoa + link rut gon
CF: spam=0.700
Phan loai: ĐƯA VÀO THƯ RÁC

Thu 2: Nguoi gui la xin OTP + link rut gon
CF: phishing=0.720, spam=0.842
Phan loai: CHẶN

Thu 3: Tieu de viet hoa + link rut gon, nguoi gui trong danh ba
CF: spam=0.000
Phan loai: CHO PHÉP

Thu 4: Gia mao ten mien ngan hang, xin OTP
CF: phishing=0.958, spam=0.910
Phan loai: CHẶN

Thu 5: Gia mao ngan hang nhung co chu ky so hop le
CF: phishing=0.958, spam=0.775
Phan loai: ĐƯA VÀO THƯ RÁC

================================================================
SO SANH NGUONG PHAN LOAI
================================================================

Thu 1: Tieu de viet hoa + link rut gon
CF spam: 0.700
Nguong cu (0.8 / 0.5): ĐƯA VÀO THƯ RÁC
Nguong moi (0.9 / 0.6): ĐƯA VÀO THƯ RÁC

Thu 2: Nguoi gui la xin OTP + link rut gon
CF spam: 0.842
Nguong cu (0.8 / 0.5): CHẶN
Nguong moi (0.9 / 0.6): ĐƯA VÀO THƯ RÁC

Thu 3: Tieu de viet hoa + link rut gon, nguoi gui trong danh ba
CF spam: 0.000
Nguong cu (0.8 / 0.5): CHO PHÉP
Nguong moi (0.9 / 0.6): CHO PHÉP

Thu 4: Gia mao ten mien ngan hang, xin OTP
CF spam: 0.910
Nguong cu (0.8 / 0.5): CHẶN
Nguong moi (0.9 / 0.6): CHẶN

Thu 5: Gia mao ngan hang nhung co chu ky so hop le
CF spam: 0.775
Nguong cu (0.8 / 0.5): ĐƯA VÀO THƯ RÁC
Nguong moi (0.9 / 0.6): ĐƯA VÀO THƯ RÁC

================================================================
PHAN LOAI NANG CAO - KIEM TRA PHISHING RIENG
================================================================

Thu 1: Tieu de viet hoa + link rut gon
```

```
============================================================
*** PHAN LOAI NANG CAO - KIEM TRA PHISHING RIENG
============================================================

Thu 1: Tieu de viet hoa + link rut gon
CF phishing: 0.0
CF spam: 0.7
Ket luan: ĐƯA VÀO THƯ RÁC

Thu 2: Nguoi gui la xin OTP + link rut gon
CF phishing: 0.72
CF spam: 0.842
Ket luan: CHẶN

Thu 3: Tieu de viet hoa + link rut gon, nguoi gui trong danh ba
CF phishing: 0.0
CF spam: 0.0
Ket luan: CHO PHÉP

Thu 4: Gia mao ten mien ngan hang, xin OTP
CF phishing: 0.958
CF spam: 0.91
Ket luan: CHẶN - NGHI PHISHING

Thu 5: Gia mao ngan hang nhung co chu ky so hop le
CF phishing: 0.958
CF spam: 0.775
Ket luan: CHẶN - NGHI PHISHING

============================================================
PHAN TICH EMAIL KHO
============================================================
Email: Thu 5: Gia mao ngan hang nhung co chu ky so hop le
CF phishing: 0.958
CF spam: 0.775
Phan loai theo spam: ĐƯA VÀO THƯ RÁC
Phan loai nang cao: CHẶN - NGHI PHISHING

Nhan xet:
- Chu ky so hop le khong dong nghia email an toan.
- Bang chung am co the lam giam diem spam.
- Neu chi kiem tra spam, he thong co the bo sot phishing.
- Can kiem tra CF phishing rieng de tang kha nang phat hien.
```

Bài 5 — Phân tích hiệu năng suy diễn tiến: vòng lặp ngây thơ so với hàng đợi + bộ đếm

Liên hệ CNTT: Forward chaining ở các bài trước quét lại toàn bộ luật sau mỗi lần có sự kiện mới — chấp nhận được với 10 luật nhưng chậm với hàng nghìn luật. Thuật toán dùng hàng đợi và bộ đếm số điều kiện chưa thỏa (ý tưởng của thuật toán Dowling–Gallier cho mệnh đề Horn) chạy tuyến tính. Đây là bài tập đánh giá độ phức tạp thuật toán trên đúng chủ đề của buổi.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
 import time
from collections import deque, defaultdict

def tao_chuoi_luat(n, nguoc=True):
     """n luật nối tiếp f0 -> f1 -> ... -> fn. nguoc=True: xếp luật theo thứ tự ngược (tình
huống xấu nhất)."""
    luat = [{"neu": [f"f{i}"], "thi": f"f{i + 1}"} for i in range(n)]
    return luat[::-1] if nguoc else luat

def fc_ngay_tho(su_kien, luat):
    facts = set(su_kien)
    so_lan_kiem_tra = 0
    thay_doi = True
    while thay_doi:
        thay_doi = False
        for r in luat:
            so_lan_kiem_tra += 1
            if r["thi"] not in facts and all(dk in facts for dk in r["neu"]):
                facts.add(r["thi"])
                thay_doi = True
    return facts, so_lan_kiem_tra

def fc_hang_doi(su_kien, luat):
```

```
    """Mỗi luật có bộ đếm 'còn thiếu bao nhiêu điều kiện'; mỗi sự kiện nhớ các luật chứa nó."""
    con_thieu = [len(set(r["neu"])) for r in luat]
    luat_chua = defaultdict(list)
    for i, r in enumerate(luat):
        for dk in set(r["neu"]):
            luat_chua[dk].append(i)
    facts = set(su_kien)
    hang_doi = deque(facts)
    so_lan_kiem_tra = 0
    while hang_doi:
        f = hang_doi.popleft()
        for i in luat_chua[f]:
            so_lan_kiem_tra += 1
            con_thieu[i] -= 1
            if con_thieu[i] == 0 and luat[i]["thi"] not in facts:
                facts.add(luat[i]["thi"])
                hang_doi.append(luat[i]["thi"])
    return facts, so_lan_kiem_tra

print(f"{'n luật':>7} | {'thứ tự':<6} | {'ngây thơ: lần kiểm tra':>22} | {'hàng đợi: lần kiểm tra':>22} | {'thời gian (ms) ngây thơ / hàng đợi':>34}")
for n in [100, 500, 1000]:
    for nguoc in [True, False]:
        luat = tao_chuoi_luat(n, nguoc)
        t0 = time.perf_counter(); kq1, d1 = fc_ngay_tho({"f0"}, luat); t1 = time.perf_counter()
        kq2, d2 = fc_hang_doi({"f0"}, luat); t2 = time.perf_counter()
        assert kq1 == kq2 and len(kq1) == n + 1      # hai thuật toán phải cho cùng kết quả
        print(f"{n:>7} | {'ngược' if nguoc else 'xuôi':<6} | {d1:>22,} | {d2:>22,} | {(t1 - t0) * 1000:>14.1f} / {(t2 - t1) * 1000:<8.2f}")
```

```
            print(f"{n:>7} | {'ngược' if nguoc else 'xuôi':<6} | {d1:>22,} | {d2:>22,} | {(t1 - t0) * 1000:>14.1f}
```

n luật	thứ tự	ngây thơ: lần kiểm tra	hàng đợi: lần kiểm tra	thời gian (ms) ngây thơ / hàng đợi
100	ngược	10,100	100	4.0 / 0.16
100	xuôi	200	100	0.1 / 0.13
500	ngược	250,500	500	77.8 / 0.70
500	xuôi	1,000	500	0.4 / 0.58
1000	ngược	1,001,000	1,000	310.4 / 1.46
1000	xuôi	2,000	1,000	0.8 / 1.25

💭 **Suy nghĩ: Khi luật được xếp ngược thứ tự, số lần kiểm tra của thuật toán ngây thơ tăng theo n hay theo n^2? Vì sao khi xếp xuôi nó lại nhanh hơn rất nhiều? Thuật toán hàng đợi tránh việc gì mà thuật toán ngây thơ lặp đi lặp lại? Với 100.000 luật, khác biệt này có ý nghĩa gì?**

Khi luật được xếp ngược, số lần kiểm tra tăng theo n hay n^2?

Số lần kiểm tra tăng theo bậc hai $O(n^2)$.

Ví dụ kết quả:

Số luật (n)	Số lần kiểm tra
100	10.100

500	250.500
1.000	1.001.000

Lý do: Khi luật được xếp ngược, thuật toán phải duyệt qua toàn bộ danh sách nhiều lần mới suy ra được kết luận tiếp theo.

Ví dụ: Muốn suy ra `f1000`, hệ thống phải lần lượt suy ra `f1`, `f2`, ... `f1000`. Do các luật bị xếp ngược, mỗi vòng lặp lại phải kiểm tra rất nhiều luật chưa thể kích hoạt.

Vì sao khi xếp xuôi thuật toán ngây thơ lại nhanh hơn?

Vì các luật đã được sắp xếp đúng theo thứ tự suy diễn:

`f0 → f1 → f2 → f3 → ...`

Khi hệ thống tìm được kết luận mới, các luật tiếp theo có thể sử dụng ngay kết luận đó trong cùng một lượt duyệt.

Vì vậy, thuật toán chỉ cần khoảng $2n$ lần kiểm tra, tương ứng độ phức tạp $O(n)$ trong ví dụ này.

Thuật toán hàng đợi tránh việc gì?

Thuật toán hàng đợi tránh việc quét lại toàn bộ danh sách luật nhiều lần.

Cụ thể:

- Mỗi sự kiện mới được đưa vào hàng đợi.
- Hệ thống chỉ kiểm tra những luật có chứa sự kiện vừa được thêm vào.
- Khi đủ điều kiện, kết luận mới tiếp tục được đưa vào hàng đợi.

Nhờ đó, mỗi quan hệ giữa sự kiện và luật chỉ cần xử lý một lần trong ví dụ này.

Với 100.000 luật, sự khác biệt có ý nghĩa gì?

Nếu tiếp tục sử dụng mô hình chuỗi luật xếp ngược:

- Thuật toán ngây thơ có thể cần khoảng 10^{10} lần kiểm tra.
- Thuật toán hàng đợi chỉ cần khoảng 100.000 lần kiểm tra.

Hàng đợi hiệu quả hơn rõ rệt

Điều này có ý nghĩa lớn trong hệ thống thực tế như SOC giám sát an ninh mạng, nơi có thể phải xử lý hàng nghìn hoặc hàng trăm nghìn luật và sự kiện.

Kết luận: Thuật toán ngây thơ phù hợp với hệ thống nhỏ, đơn giản. Thuật toán hàng đợi phù hợp hơn với hệ chuyên gia lớn vì giảm số lần kiểm tra dư thừa, tiết kiệm thời gian xử lý và tăng khả năng mở rộng.

Các con số trên là theo cấu trúc chuỗi luật của bài thực hành, không phải cam kết thời gian thực tế cho mọi hệ thống có 100.000 luật.

🔧 Cải tiến: Thêm n = 3000 vào danh sách (cẩn thận: thuật toán ngây thơ sẽ chậm hơn nhiều) và vẽ biểu đồ số lần kiểm tra theo n cho cả hai thuật toán (matplotlib). Sau đó thử tạo bộ luật có 2 điều kiện mỗi luật (ví dụ f_i VÀ g_i → f_(i+1)) và kiểm tra thuật toán hàng đợi vẫn đúng.

```
import time
import matplotlib.pyplot as plt
from collections import deque, defaultdict

# ====================================================
# 1. TAO CHUOI LUAT MOT DIEU KIEN
# ====================================================

def tao_chuoi_luat(n, nguoc=True):
    luat = [
        {"neu": [f"f{i}"], "thi": f"f{i+1}"}
        for i in range(n)
    ]
    return luat[::-1] if nguoc else luat


# ====================================================
# 2. THUAT TOAN FORWARD CHAINING NGAY THO
# ====================================================

def fc_ngay_tho(su_kien, luat):
    facts = set(su_kien)
    so_lan_kiem_tra = 0

    thay_doi = True

    while thay_doi:
        thay_doi = False

        for r in luat:
            so_lan_kiem_tra += 1

            if (r["thi"] not in facts
                    and all(dk in facts for dk in r["neu"])):

                facts.add(r["thi"])
                thay_doi = True

    return facts, so_lan_kiem_tra


# ====================================================
# 3. THUAT TOAN FORWARD CHAINING HANG DOI
# ====================================================

def fc_hang_doi(su_kien, luat):
    con_thieu = [len(set(r["neu"])) for r in luat]

    luat_chua = defaultdict(list)

    for i, r in enumerate(luat):
        for dk in set(r["neu"]):
            luat_chua[dk].append(i)
```

```
    facts = set(su_kien)
    hang_doi = deque(facts)

    so_lan_kiem_tra = 0

    while hang_doi:
        f = hang_doi.popleft()

        for i in luat_chua[f]:
            so_lan_kiem_tra += 1
            con_thieu[i] -= 1

            if con_thieu[i] == 0 and luat[i]["thi"] not in facts:
                facts.add(luat[i]["thi"])
                hang_doi.append(luat[i]["thi"])

    return facts, so_lan_kiem_tra


# ==================================================
# 4. THU NGHIEM VOI 100, 500, 1000, 3000 LUAT
# ==================================================

ds_n = [100, 500, 1000, 3000]

ket_qua_nguoc = []
ket_qua_xuoi = []

print("SO SANH HAI THUAT TOAN")
print("-" * 85)

print(
    f"{'n luật':>7} | {'Thứ tự':<7} | "
    f"{'Ngây thơ':>15} | {'Hàng đợi':>15} | "
    f"{'Thời gian (ms)':>20}"
)

for n in ds_n:
    for nguoc in [True, False]:

        luat = tao_chuoi_luat(n, nguoc)

        t0 = time.perf_counter()
        kq1, d1 = fc_ngay_tho({"f0"}, luat)
        t1 = time.perf_counter()

        kq2, d2 = fc_hang_doi({"f0"}, luat)
        t2 = time.perf_counter()

        assert kq1 == kq2
        assert len(kq1) == n + 1

        print(
            f"{n:>7} | "
            f"{'Ngược' if nguoc else 'Xuôi':<7} | "
            f"{d1:>15,} | "
            f"{d2:>15,} | "
            f"{(t1-t0)*1000:>8.2f} / {(t2-t1)*1000:<8.2f}"
        )
```

```
        if nguoc:
            ket_qua_nguoc.append((d1, d2))
        else:
            ket_qua_xuoi.append((d1, d2))


# ====================================================
# 5. VE BIEU DO SO LAN KIEM TRA
# ====================================================

ngay_tho_nguoc = [x[0] for x in ket_qua_nguoc]
hang_doi_nguoc = [x[1] for x in ket_qua_nguoc]

ngay_tho_xuoi = [x[0] for x in ket_qua_xuoi]
hang_doi_xuoi = [x[1] for x in ket_qua_xuoi]

plt.figure(figsize=(10, 6))

plt.plot(
    ds_n, ngay_tho_nguoc,
    marker="o", label="Ngây thơ - luật ngược"
)

plt.plot(
    ds_n, hang_doi_nguoc,
    marker="o", label="Hàng đợi - luật ngược"
)

plt.plot(
    ds_n, ngay_tho_xuoi,
    marker="s", label="Ngây thơ - luật xuôi"
)

plt.plot(
    ds_n, hang_doi_xuoi,
    marker="s", label="Hàng đợi - luật xuôi"
)

plt.xlabel("Số lượng luật (n)")
plt.ylabel("Số lần kiểm tra")
plt.title("So sánh số lần kiểm tra của hai thuật toán")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


# ====================================================
# 6. TAO BO LUAT CO HAI DIEU KIEN AND
# f_i AND g_i -> f_(i+1)
# ====================================================

def tao_luat_hai_dieu_kien(n, nguoc=True):

    luat = [
        {
            "neu": [f"f{i}", f"g{i}"],
            "thi": f"f{i+1}"
        }
```

```
        for i in range(n)
    ]

    return luat[::-1] if nguoc else luat


# Tạo 100 luật hai điều kiện để kiểm tra
n_test = 100

luat_2_dk = tao_luat_hai_dieu_kien(n_test, nguoc=True)

# Ban đầu có f0 và toàn bộ g0, g1, ..., g99
su_kien_2_dk = {"f0"}

for i in range(n_test):
    su_kien_2_dk.add(f"g{i}")


# ====================================================
# 7. KIEM TRA HAI THUAT TOAN VOI LUAT HAI DIEU KIEN
# ====================================================

kq_ngay_tho, dem_ngay_tho = fc_ngay_tho(
    su_kien_2_dk, luat_2_dk
)

kq_hang_doi, dem_hang_doi = fc_hang_doi(
    su_kien_2_dk, luat_2_dk
)

print("\n" + "=" * 60)
print("KIEM TRA LUAT CO HAI DIEU KIEN AND")
print("=" * 60)

print("Số luật:", n_test)
print("Sự kiện ban đầu:", len(su_kien_2_dk))

print("Số sự kiện sau suy diễn:", len(kq_hang_doi))
print("Số lần kiểm tra - ngây thơ:", dem_ngay_tho)
print("Số lần kiểm tra - hàng đợi:", dem_hang_doi)

print("Hai thuật toán cho kết quả giống nhau:",
      kq_ngay_tho == kq_hang_doi)

# Kiểm tra tất cả f0 đến f100 đều được suy ra
du_kien = {f"f{i}" for i in range(n_test + 1)}

assert du_kien.issubset(kq_hang_doi)
assert kq_ngay_tho == kq_hang_doi

print("\nKẾT QUẢ: THUẬT TOÁN HÀNG ĐỢI HOẠT ĐỘNG ĐÚNG!")


# ====================================================
# 8. IN MOT SO KET LUAN SUY DIEN
# ====================================================

print("\nMột số sự kiện suy diễn:")
for i in range(0, n_test + 1, 10):
    print(f"f{i}", end=" -> ")
```

```
print("HOÀN TẤT")
```

```
print("HOÀN TẤT")
```

```
   100 | Ngược   |         10,100 |           100 |    20.63 / 0.32
   100 | Xuôi    |            200 |           100 |     4.15 / 0.26
   500 | Ngược   |        250,500 |           500 |   254.82 / 1.11
   500 | Xuôi    |          1,000 |           500 |     0.77 / 6.86
  1000 | Ngược   |      1,001,000 |         1,000 |  1719.48 / 2.18
  1000 | Xuôi    |          2,000 |         1,000 |     1.77 / 2.14
  3000 | Ngược   |      9,003,000 |         3,000 |  3308.80 / 4.18
  3000 | Xuôi    |          6,000 |         3,000 |     2.74 / 3.91
```




```
============================================================
KIEM TRA LUAT CO HAI DIEU KIEN AND
============================================================
So luat: 100
Su kien ban dau: 101
So su kien sau suy dien: 201
So lan kiem tra - ngay tho: 10100
So lan kiem tra - hang doi: 200
Hai thuat toan cho ket qua giong nhau: True

KET QUA: THUAT TOAN HANG DOI HOAT DONG DUNG!

Mot so su kien suy dien:
f0 -> f10 -> f20 -> f30 -> f40 -> f50 -> f60 -> f70 -> f80 -> f90 -> f100 -> HOAN TAT
```

TIẾT 4 (BỔ SUNG B) — 5 BÀI ỨNG DỤNG ĐA LĨNH VỰC (BÀI 6–10)

Năm bài dưới đây đưa hệ chuyên gia và suy diễn bằng luật sang năm lĩnh vực khác nhau: y tế, nông nghiệp thông minh, tài chính, giáo dục và một động cơ suy diễn dùng chung cho mọi lĩnh vực. Là sinh viên CNTT, bạn đóng vai kỹ sư tri thức: nhận luật từ chuyên gia lĩnh vực và biến thành chương trình chạy được. Mỗi bài có code chạy sẵn, câu hỏi Suy nghĩ và yêu cầu Cải tiến.

Bài 6 — Y tế: hệ chuyên gia sàng lọc triệu chứng và chọn mức khẩn cấp khi các luật xung đột

Liên hệ đa lĩnh vực (Y tế): các ứng dụng sức khỏe số thường có bước sàng lọc triệu chứng bằng luật để hướng dẫn người dùng nên tự chăm sóc, đặt lịch khám hay gọi cấp cứu. Điểm kỹ thuật: số đo (nhiệt độ, tuổi) được rời rạc hóa thành sự kiện, và khi nhiều luật cùng khớp thì chọn mức khẩn cấp CAO NHẤT cho an toàn. Chỉ mang tính minh họa kỹ thuật, không thay thế chẩn đoán y khoa.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
# ---- Hệ chuyên gia sàng lọc triệu chứng (chỉ mang tính hướng dẫn, KHÔNG thay thế chẩn đoán của bác sĩ)
MUC = {1: "Tự chăm sóc tại nhà", 2: "Nên đặt lịch khám", 3: "Cần đi khám trong ngày", 4: "CẤP CỨU - gọi 115"}
```

```
def roi_rac_hoa(bn):
    """Biến số liệu thành sự kiện rời rạc để hệ luật xử lý"""
    s = set(bn["trieu_chung"])
    if bn["nhiet_do"] >= 38.5:
        s.add("sot_cao")
    if bn["so_ngay_ho"] >= 14:
        s.add("ho_keo_dai")
    if bn["tuoi"] < 5:
        s.add("tre_nho")
    return s

LUAT = [
       {"ten": "Y1", "neu": ["sot_cao", "ho", "dau_hong"],                "thi": "nghi_cum",
"muc": 1},
    {"ten": "Y2", "neu": ["sot_cao", "kho_tho"],               "thi": "nghi_nhiem_trung_ho_hap",
"muc": 3},
     {"ten": "Y3", "neu": ["dau_nguc", "kho_tho"],                  "thi": "nghi_van_de_tim_phoi",
"muc": 4},
      {"ten": "Y4", "neu": ["sot_cao", "phat_ban", "cung_co"],      "thi": "nghi_viem_mang_nao",
"muc": 4},
    {"ten": "Y5", "neu": ["ho_keo_dai"],                            "thi": "can_kham_chuyen_khoa",
"muc": 2},
      {"ten": "Y6", "neu": ["tre_nho", "sot_cao"],                     "thi": "tre_nho_sot_cao",
"muc": 3},
]

def sang_loc(ten, bn):
    facts = roi_rac_hoa(bn)
    kich_hoat = [r for r in LUAT if all(dk in facts for dk in r["neu"])]
    print(f"=== {ten} ===")
    if not kich_hoat:
        print("  Chưa có luật nào khớp -> theo dõi thêm, nếu nặng hơn hãy đi khám.\n")
        return 1
    for r in kich_hoat:
        print(f"  {r['ten']}: {r['thi']} (mức {r['muc']})")
    muc_cao_nhat = max(r["muc"] for r in kich_hoat)       # xung đột -> chọn mức khẩn cấp CAO NHẤT
cho an toàn
    print(f"  => Khuyến nghị: {MUC[muc_cao_nhat]}\n")
    return muc_cao_nhat

sang_loc("Ca  A",  {"tuoi":  25,  "nhiet_do":  38.8,  "so_ngay_ho":  2,  "trieu_chung":  ["ho",
"dau_hong"]})
sang_loc("Ca  B",  {"tuoi":  60,  "nhiet_do":  37.0,  "so_ngay_ho":  1,  "trieu_chung":  ["dau_nguc",
"kho_tho"]})
sang_loc("Ca C", {"tuoi": 3, "nhiet_do": 39.0, "so_ngay_ho": 1, "trieu_chung": ["ho"]})
sang_loc("Ca D", {"tuoi": 40, "nhiet_do": 36.8, "so_ngay_ho": 20, "trieu_chung": ["ho"]})
sang_loc("Ca E", {"tuoi": 30, "nhiet_do": 36.6, "so_ngay_ho": 0, "trieu_chung": ["met_moi"]})
```

```
=== Ca A ===
  Y1: nghi_cum (mức 1)
  => Khuyến nghị: Tự chăm sóc tại nhà

=== Ca B ===
  Y3: nghi_van_de_tim_phoi (mức 4)
  => Khuyến nghị: CẤP CỨU - gọi 115

=== Ca C ===
  Y6: tre_nho_sot_cao (mức 3)
  => Khuyến nghị: Cần đi khám trong ngày

=== Ca D ===
  Y5: can_kham_chuyen_khoa (mức 2)
  => Khuyến nghị: Nên đặt lịch khám

=== Ca E ===
  Chưa có luật nào khớp -> theo dõi thêm, nếu nặng hơn hãy đi khám.

1
```

💭 **Suy nghĩ: Vì sao khi nhiều luật cùng khớp, hệ thống chọn mức khẩn cấp cao nhất thay vì mức thấp nhất hoặc mức trung bình? Ca E không khớp luật nào, hệ thống nên trả lời thế nào cho an toàn? Bước "rời rạc hóa" (ví dụ nhiệt độ ≥ 38,5 thành sot_cao) có ưu và nhược điểm gì so với dùng số liệu trực tiếp?**

Vì sao hệ thống chọn mức khẩn cấp cao nhất khi nhiều luật cùng khớp?

Vì đây là hệ thống sàng lọc sức khỏe, mục tiêu quan trọng nhất là đảm bảo an toàn và tránh bỏ sót tình trạng nguy hiểm.

Ví dụ, nếu một bệnh nhân đồng thời khớp:

- Y1: Nghi cúm – mức 1.
- Y3: Nghi vấn đề tim phổi – mức 4.

Hệ thống phải chọn mức 4 (cấp cứu), thay vì chọn mức thấp hơn hoặc lấy trung bình.

Nếu chọn mức thấp, bệnh nhân có thể bị đánh giá thấp mức độ nguy hiểm và chậm được cấp cứu.

Ca E không khớp luật nào, hệ thống nên trả lời thế nào cho an toàn?

Ca E chỉ có triệu chứng `met_moi`, không khớp với bất kỳ luật nào.

Hệ thống không nên tự kết luận bệnh nhân khỏe mạnh hoặc chắc chắn không có bệnh.

Cách trả lời an toàn hơn:

- Chưa đủ dữ kiện để đưa ra nhận định.
- Theo dõi tình trạng sức khỏe và bổ sung thông tin triệu chứng.
- Nếu triệu chứng kéo dài, nặng lên hoặc xuất hiện dấu hiệu bất thường thì nên đi khám.
- Nếu có khó thở, đau ngực dữ dội, bất tỉnh hoặc dấu hiệu nguy hiểm khác thì cần gọi cấp cứu 115.

Lưu ý: Không khớp luật không đồng nghĩa với không có bệnh.

Ưu và nhược điểm của bước rời rạc hóa

Ví dụ:

```
None
if bn["nhiet_do"] >= 38.5:    s.add("sot_cao")
```

Tiêu chí	Rời rạc hóa	Dùng số liệu trực tiếp
Cách xử lý	Chuyển số liệu thành sự kiện	Sử dụng giá trị thực tế
Ưu điểm	Đơn giản, dễ xây dựng luật IF-THEN	Giữ được thông tin chi tiết

Nhược điểm	Mất một phần thông tin	Luật phức tạp hơn
Ví dụ	38,5°C và 40°C đều là `sot_cao`	Phân biệt được 38,5°C và 40°C

Ưu điểm của rời rạc hóa:

- Giúp hệ chuyên gia dễ hiểu và xử lý dữ liệu.
- Luật IF-THEN đơn giản, dễ kiểm tra.
- Dễ xây dựng các nhóm triệu chứng.

Nhược điểm:

- Mất độ chính xác của dữ liệu ban đầu.
- Các giá trị gần ngưỡng có thể bị phân loại khác nhau dù chênh lệch rất nhỏ.
- Không thể hiện đầy đủ mức độ nghiêm trọng của triệu chứng.

Kết luận

Hệ thống chọn mức khẩn cấp cao nhất để ưu tiên an toàn. Khi không có luật nào khớp, cần thông báo chưa đủ dữ kiện thay vì kết luận bệnh nhân bình thường. Rời rạc hóa giúp hệ chuyên gia đơn giản và dễ xây dựng, nhưng có thể làm mất thông tin quan trọng so với sử dụng số liệu trực tiếp.

🔧 Cải tiến: Thêm 2 luật mới của bạn (ví dụ: nôn nhiều + tiêu chảy + tre_nho → mất nước, mức 3; đau bụng dữ dội + sốt → nghi viêm ruột thừa, mức 3) và thêm một ca bệnh mới để kiểm tra. Sau đó thêm Ca F (tuổi 28, nhiệt độ 38,2, triệu chứng ho và đau họng), chạy trước và sau khi đổi ngưỡng sốt cao từ 38,5 xuống 38,0 và nhận xét kết quả thay đổi thế nào, ngưỡng nên do ai quyết định.

```
# ================================================
# HE CHUYEN GIA SANG LOC TRIEU CHUNG
# CHI MANG TINH CHAT HOC TAP
# ================================================

MUC = {
  1: "Tự chăm sóc tại nhà",
  2: "Nên đặt lịch khám",
  3: "Cần đi khám trong ngày",
  4: "CẤP CỨU - gọi 115"
}
```

```
# ====================================================
# 1. ROI RAC HOA DU LIEU
# ====================================================

def roi_rac_hoa(bn, nguong_sot=38.5):
    s = set(bn["trieu_chung"])

    if bn["nhiet_do"] >= nguong_sot:
        s.add("sot_cao")

    if bn["nhiet_do"] >= 38.0:
        s.add("sot")

    if bn["so_ngay_ho"] >= 14:
        s.add("ho_keo_dai")

    if bn["tuoi"] < 5:
        s.add("tre_nho")

    return s


# ====================================================
# 2. KHAI BAO 8 LUAT
# ====================================================

LUAT = [
    {
        "ten": "Y1",
```

```
    "neu": ["sot_cao", "ho", "dau_hong"],
    "thi": "nghi_cum",
    "muc": 1
},
{
    "ten": "Y2",
    "neu": ["sot_cao", "kho_tho"],
    "thi": "nghi_nhiem_trung_ho_hap",
    "muc": 3
},
{
    "ten": "Y3",
    "neu": ["dau_nguc", "kho_tho"],
    "thi": "nghi_van_de_tim_phoi",
    "muc": 4
},
{
    "ten": "Y4",
    "neu": ["sot_cao", "phat_ban", "cung_co"],
    "thi": "nghi_viem_mang_nao",
    "muc": 4
},
{
    "ten": "Y5",
    "neu": ["ho_keo_dai"],
    "thi": "can_kham_chuyen_khoa",
    "muc": 2
},
{
    "ten": "Y6",
```

```
    "neu": ["tre_nho", "sot_cao"],
    "thi": "tre_nho_sot_cao",
    "muc": 3
  },

  # Y7: Nghi mat nuoc
  {
    "ten": "Y7",
    "neu": ["non_nhieu", "tieu_chay", "tre_nho"],
    "thi": "nghi_mat_nuoc",
    "muc": 3
  },

  # Y8: Nghi viem ruot thua
  {
    "ten": "Y8",
    "neu": ["dau_bung_du_doi", "sot"],
    "thi": "nghi_viem_ruot_thua",
    "muc": 3
  }
]



# ================================================
# 3. HAM SANG LOC
# ================================================

def sang_loc(ten, bn, nguong_sot=38.5):
  facts = roi_rac_hoa(bn, nguong_sot)
```

```
    kich_hoat = [
        r for r in LUAT
        if all(dk in facts for dk in r["neu"])
    ]

    print(f"=== {ten} ===")
    print("Sự kiện:", sorted(facts))

    if not kich_hoat:
        print("Chưa có luật nào khớp.")
        print("Chưa đủ dữ kiện để đưa ra nhận định.")
        print("Theo dõi triệu chứng và đi khám nếu tình trạng kéo dài hoặc nặng lên.\n")
        return 0

    for r in kich_hoat:
        print(f"{r['ten']}: {r['thi']} (mức {r['muc']})")

    muc_cao_nhat = max(r["muc"] for r in kich_hoat)

    print(f"=> Khuyến nghị: {MUC[muc_cao_nhat]}\n")

    return muc_cao_nhat



# ====================================================
# 4. CAC CA BENH THU NGHIEM
# ====================================================

ca_benh = {
    "Ca A": {
```

```
    "tuoi": 25,
    "nhiet_do": 38.8,
    "so_ngay_ho": 2,
    "trieu_chung": ["ho", "dau_hong"]
},

"Ca B": {
    "tuoi": 60,
    "nhiet_do": 37.0,
    "so_ngay_ho": 1,
    "trieu_chung": ["dau_nguc", "kho_tho"]
},

"Ca C": {
    "tuoi": 3,
    "nhiet_do": 39.0,
    "so_ngay_ho": 1,
    "trieu_chung": ["ho"]
},

"Ca D": {
    "tuoi": 40,
    "nhiet_do": 36.8,
    "so_ngay_ho": 20,
    "trieu_chung": ["ho"]
},

"Ca E": {
    "tuoi": 30,
    "nhiet_do": 36.6,
```

```
    "so_ngay_ho": 0,
    "trieu_chung": ["met_moi"]
  },

  # Ca moi: nghi mat nuoc
  "Ca G": {
    "tuoi": 3,
    "nhiet_do": 38.2,
    "so_ngay_ho": 0,
    "trieu_chung": [
      "non_nhieu",
      "tieu_chay"
    ]
  }
}



# ================================================
# 5. CHAY CAC CA BENH
# ================================================


print("=" * 55)
print("KET QUA SANG LOC CAC CA BENH")
print("=" * 55)


for ten, bn in ca_benh.items():
  sang_loc(ten, bn, nguong_sot=38.5)



# ================================================
```

```
# 6. THU NGHIEM CA F VOI HAI NGUONG SOT
# ==================================================

ca_F = {
    "tuoi": 28,
    "nhiet_do": 38.2,
    "so_ngay_ho": 2,
    "trieu_chung": ["ho", "dau_hong"]
}

print("=" * 55)
print("SO SANH CA F VOI HAI NGUONG SOT")
print("=" * 55)

print("\n--- Truoc khi thay doi: nguong 38.5 do C ---")
kq_cu = sang_loc("Ca F", ca_F, nguong_sot=38.5)

print("--- Sau khi thay doi: nguong 38.0 do C ---")
kq_moi = sang_loc("Ca F", ca_F, nguong_sot=38.0)

print("Kết quả trước:", MUC.get(kq_cu, "Chưa đủ dữ kiện"))
print("Kết quả sau:", MUC.get(kq_moi, "Chưa đủ dữ kiện"))

if kq_cu != kq_moi:
    print("=> Kết quả sàng lọc đã thay đổi.")
else:
    print("=> Kết quả sàng lọc không thay đổi.")
```

```
...  ============================================================
     KET QUA SANG LOC CAC CA BENH
     ============================================================
     === Ca A ===
     Sự kiện: ['dau_hong', 'ho', 'sot', 'sot_cao']
     Y1: nghi_cum (mức 1)
     => Khuyến nghị: Tự chăm sóc tại nhà

     === Ca B ===
     Sự kiện: ['dau_nguc', 'kho_tho']
     Y3: nghi_van_de_tim_phoi (mức 4)
     => Khuyến nghị: CẤP CỨU - gọi 115

     === Ca C ===
     Sự kiện: ['ho', 'sot', 'sot_cao', 'tre_nho']
     Y6: tre_nho_sot_cao (mức 3)
     => Khuyến nghị: Cần đi khám trong ngày

     === Ca D ===
     Sự kiện: ['ho', 'ho_keo_dai']
     Y5: can_kham_chuyen_khoa (mức 2)
     => Khuyến nghị: Nên đặt lịch khám

     === Ca E ===
     Sự kiện: ['met_moi']
     Chưa có luật nào khớp.
     Chưa đủ dữ kiện để đưa ra nhận định.
     Theo dõi triệu chứng và đi khám nếu tình trạng kéo dài hoặc nặng lên.

     === Ca G ===
     Sự kiện: ['non_nhieu', 'sot', 'tieu_chay', 'tre_nho']
     Y7: nghi_mat_nuoc (mức 3)
     => Khuyến nghị: Cần đi khám trong ngày

     ============================================================
     SO SANH CA F VOI HAI NGUONG SOT
     ============================================================

     --- Truoc khi thay doi: nguong 38.5 do C ---
     === Ca F ===
     Sự kiện: ['dau_hong', 'ho', 'sot']
     Chưa có luật nào khớp.
     Chưa đủ dữ kiện để đưa ra nhận định.
     Theo dõi triệu chứng và đi khám nếu tình trạng kéo dài hoặc nặng lên.

     --- Sau khi thay doi: nguong 38.0 do C ---
     === Ca F ===
     Sự kiện: ['dau_hong', 'ho', 'sot', 'sot_cao']
     Y1: nghi_cum (mức 1)
     => Khuyến nghị: Tự chăm sóc tại nhà

     Kết quả trước: Chưa đủ dữ kiện
     Kết quả sau: Tự chăm sóc tại nhà
     => Kết quả sàng lọc đã thay đổi.
```

Bài 7 — Nông nghiệp thông minh: từ cảm biến IoT đến khuyến nghị tưới và bón phân

Liên hệ đa lĩnh vực (Nông nghiệp / IoT): nhà vườn thông minh thu số liệu cảm biến liên tục. Hệ luật biến số đo thành sự kiện (đất khô, trời nóng, đất chua...) rồi suy diễn tiến nhiều tầng ra hành động. Có luật chồng lấn (cần tưới nhưng sắp có mưa) nên hệ luật cần các luật điều phối. (Số liệu cảm biến giả lập.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
    # ---- Nông nghiệp thông minh: số đo cảm biến IoT -> sự kiện rời rạc -> hệ luật -> khuyến
nghị
def roi_rac_hoa(doc):
    s = set()
      s.add("dat_kho" if doc["do_am_dat"] < 30 else "dat_uot" if doc["do_am_dat"] > 75 else
"dat_vua")
    if doc["nhiet_do"] > 35:
        s.add("nong")
    if doc["do_am_kk"] > 85:
        s.add("khong_khi_am")
    if doc["ph_dat"] < 5.5:
        s.add("dat_chua")
    if doc["dam"] < 20:
```

```
        s.add("thieu_dam")
    if doc["mua_ngay_mai"]:
        s.add("co_mua")
    return s

LUAT = [
    {"ten": "N1", "neu": ["dat_kho", "nong"],                "thi": "tuoi_ngay_lap_tuc"},
    {"ten": "N2", "neu": ["dat_kho"],                        "thi": "can_tuoi"},
    {"ten": "N3", "neu": ["can_tuoi", "co_mua"],             "thi": "hoan_tuoi_cho_mua"},
    {"ten": "N4", "neu": ["dat_uot", "khong_khi_am"],       "thi": "nguy_co_nam_benh"},
         {"ten":  "N5",  "neu":  ["nguy_co_nam_benh"],                                  "thi":
"khuyen_nghi_phun_thuoc_nam"},
    {"ten": "N6", "neu": ["dat_chua"],                      "thi": "khuyen_nghi_bon_voi"},
    {"ten": "N7", "neu": ["thieu_dam", "dat_vua"],         "thi": "khuyen_nghi_bon_dam"},
]

def suy_dien(su_kien, luat):
    facts = set(su_kien)
    thay_doi = True
    while thay_doi:
        thay_doi = False
        for r in luat:
            if r["thi"] not in facts and all(dk in facts for dk in r["neu"]):
                facts.add(r["thi"])
                thay_doi = True
    return facts

# ---- 6 lần đọc cảm biến trong ngày (dữ liệu giả lập)
doc_cam_bien = [
    {"gio": "06:00", "do_am_dat": 45, "nhiet_do": 24, "do_am_kk": 90, "ph_dat": 6.2, "dam": 35,
"mua_ngay_mai": False},
    {"gio": "10:00", "do_am_dat": 28, "nhiet_do": 33, "do_am_kk": 60, "ph_dat": 6.2, "dam": 35,
"mua_ngay_mai": False},
    {"gio": "13:00", "do_am_dat": 22, "nhiet_do": 37, "do_am_kk": 45, "ph_dat": 6.2, "dam": 18,
"mua_ngay_mai": False},
    {"gio": "15:00", "do_am_dat": 25, "nhiet_do": 34, "do_am_kk": 50, "ph_dat": 5.2, "dam": 18,
"mua_ngay_mai": True},
    {"gio": "19:00", "do_am_dat": 80, "nhiet_do": 27, "do_am_kk": 92, "ph_dat": 5.2, "dam": 18,
"mua_ngay_mai": True},
    {"gio": "22:00", "do_am_dat": 55, "nhiet_do": 25, "do_am_kk": 70, "ph_dat": 6.0, "dam": 15,
"mua_ngay_mai": False},
]

for doc in doc_cam_bien:
    sk = roi_rac_hoa(doc)
    ket_qua = suy_dien(sk, LUAT)
        hanh_dong = sorted(f for f in ket_qua if f.startswith(("can_", "tuoi_", "hoan_",
"khuyen_nghi_")))
    print(f"{doc['gio']}  sự kiện: {sorted(sk)}")
    print(f"       hành động: {hanh_dong if hanh_dong else 'không cần can thiệp'}")
```

```
06:00  sự kiện: ['dat_vua', 'khong_khi_am']
       hành động: không cần can thiệp
10:00  sự kiện: ['dat_kho']
       hành động: ['can_tuoi']
13:00  sự kiện: ['dat_kho', 'nong', 'thieu_dam']
       hành động: ['can_tuoi', 'tuoi_ngay_lap_tuc']
15:00  sự kiện: ['co_mua', 'dat_chua', 'dat_kho', 'thieu_dam']
       hành động: ['can_tuoi', 'hoan_tuoi_cho_mua', 'khuyen_nghi_bon_voi']
19:00  sự kiện: ['co_mua', 'dat_chua', 'dat_uot', 'khong_khi_am', 'thieu_dam']
       hành động: ['khuyen_nghi_bon_voi', 'khuyen_nghi_phun_thuoc_nam']
22:00  sự kiện: ['dat_vua', 'thieu_dam']
       hành động: ['khuyen_nghi_bon_dam']
```

💭 **Suy nghĩ: Ở 15:00 hệ thống đề xuất hoãn tưới chờ mưa nhưng vẫn kèm "can_tuoi": hai sự kiện này có mâu thuẫn không, và hành động cuối cùng nên là gì? Ở 19:00 vì sao suy ra phun thuốc nấm qua hai tầng luật? Số đo nào trong dữ liệu được rời rạc hóa theo ngưỡng, và ngưỡng nên do ai (kỹ sư hay chuyên gia nông học) quyết định?**

Ở thời điểm 15:00, hệ thống đưa ra hai hành động là `can_tuoi` và `hoan_tuoi_cho_mua`. Đây là hai hành động có sự xung đột về mặt thực hiện, bởi vì một hành động yêu cầu tưới nước, trong khi hành động còn lại yêu cầu hoãn tưới để chờ mưa. Nguyên nhân là luật N2 được kích hoạt khi độ ẩm đất dưới 30%, tạo ra kết luận `can_tuoi`. Sau đó, luật N3 tiếp tục sử dụng kết luận này kết hợp với sự kiện `co_mua` để suy ra `hoan_tuoi_cho_mua`. Tuy nhiên, chương trình chỉ bổ sung kết luận mới vào tập sự kiện mà không loại bỏ kết luận cũ, nên cả hai hành động cùng xuất hiện trong kết quả. Theo dữ liệu tại thời điểm 15:00, hành động hợp lý hơn là hoãn tưới và tiếp tục theo dõi tình trạng đất cũng như lượng mưa thực tế, thay vì vừa tưới vừa chờ mưa. Để giải quyết vấn đề này, hệ thống cần bổ sung cơ chế ưu tiên hành động, trong đó khi có kết luận hoãn tưới thì loại bỏ hành động tưới thông thường. Tuy nhiên, quyết định cuối cùng trong thực tế còn phải dựa trên lượng mưa dự kiến và tình trạng cây trồng.

Ở thời điểm 19:00, hệ thống suy ra khuyến nghị phun thuốc nấm thông qua hai tầng luật suy diễn. Cụ thể, độ ẩm đất đạt 80%, lớn hơn ngưỡng 75%, nên được chuyển thành sự kiện `dat_uot`; độ ẩm không khí đạt 92%, lớn hơn ngưỡng 85%, nên được chuyển thành sự kiện `khong_khi_am`. Khi cả hai sự kiện này xuất hiện, luật N4 được kích hoạt và tạo ra kết luận `nguy_co_nam_benh`. Tiếp theo, kết luận này trở thành dữ kiện đầu vào cho luật N5, từ đó hệ thống suy ra `khuyen_nghi_phun_thuoc_nam`. Đây là ví dụ về suy diễn tiến nhiều tầng (Forward Chaining), trong đó kết luận của luật trước tiếp tục được sử dụng để kích hoạt luật sau. Tuy nhiên, kết quả này chỉ thể hiện nguy cơ xuất hiện nấm bệnh do điều kiện môi trường thuận lợi, không có nghĩa cây trồng chắc chắn đã bị nhiễm nấm. Vì vậy, cần kiểm tra tình trạng cây trước khi quyết định sử dụng thuốc bảo vệ thực vật.

Trong chương trình, các số đo cảm biến được rời rạc hóa theo ngưỡng gồm độ ẩm đất, nhiệt độ, độ ẩm không khí, độ pH của đất và hàm lượng đạm. Cụ thể, độ ẩm đất dưới 30% được xác định là `dat_kho`, trên 75% là `dat_uot`, còn từ 30% đến 75% là `dat_vua`; nhiệt độ trên 35°C được xác định là `nong`; độ ẩm không khí trên 85% được xác định là `khong_khi_am`; pH đất dưới 5,5 được xác định là `dat_chua`; hàm lượng đạm dưới 20 được xác định là `thieu_dam`. Riêng dữ liệu `mua_ngay_mai` là giá trị Boolean, được chuyển thành sự kiện `co_mua` khi có dự báo mưa, chứ không phải số đo được phân loại theo ngưỡng.

Về việc lựa chọn ngưỡng, chuyên gia nông học nên là người quyết định về mặt chuyên môn, bởi vì mỗi loại cây trồng, loại đất và giai đoạn sinh trưởng có nhu cầu về nước, nhiệt độ, độ pH và dinh dưỡng khác nhau. Kỹ sư IoT và kỹ sư phần mềm có trách nhiệm triển khai các ngưỡng đó vào chương trình, xử lý dữ liệu cảm biến và xây dựng các luật suy diễn. Ngoài ra, người vận hành trang trại có thể cung cấp dữ liệu thực tế để kiểm tra và điều chỉnh hệ thống. Như vậy, ngưỡng không nên do kỹ sư tự đặt tùy ý mà cần dựa trên kiến thức chuyên môn của chuyên gia nông học và kết quả khảo nghiệm thực tế.

🔧 Cải tiến: Thêm luật cảnh báo sương muối (nhiet_do < 8 → canh_bao_suong_muoi → khuyen_nghi_che_phu), thêm một lần đọc cảm biến lúc 04:00 với nhiet_do = 6 để kiểm tra. Sau đó sửa luật N2 để không còn đề xuất can_tuoi khi đã có co_mua (gợi ý: đưa điều kiện khong_co_mua vào luật, hoặc xử lý sau suy diễn).

```
def roi_rac_hoa(doc):
    s = set()

    # Do am dat
```

```
s.add(
    "dat_kho" if doc["do_am_dat"] < 30
    else "dat_uot" if doc["do_am_dat"] > 75
    else "dat_vua"
)

# Nhiet do
if doc["nhiet_do"] > 35:
    s.add("nong")

if doc["nhiet_do"] < 8:
    s.add("canh_bao_suong_muoi")

# Do am khong khi
if doc["do_am_kk"] > 85:
    s.add("khong_khi_am")

# pH dat
if doc["ph_dat"] < 5.5:
    s.add("dat_chua")

# Ham luong dam
if doc["dam"] < 20:
    s.add("thieu_dam")

# Du bao mua
if doc["mua_ngay_mai"]:
    s.add("co_mua")
else:
    s.add("khong_co_mua")
```

```
    return s



# ==============================
# KHAI BAO CAC LUAT
# ==============================


LUAT = [
    # N1: Dat kho va troi nong
    {
        "ten": "N1",
        "neu": ["dat_kho", "nong"],
        "thi": "tuoi_ngay_lap_tuc"
    },

    # N2: Chi tuoi khi dat kho va khong co mua
    {
        "ten": "N2",
        "neu": ["dat_kho", "khong_co_mua"],
        "thi": "can_tuoi"
    },

    # N3: Dat kho nhung ngay mai co mua
    {
        "ten": "N3",
        "neu": ["dat_kho", "co_mua"],
        "thi": "hoan_tuoi_cho_mua"
    },
```

```
# N4: Nguy co nam benh
{
    "ten": "N4",
    "neu": ["dat_uot", "khong_khi_am"],
    "thi": "nguy_co_nam_benh"
},

# N5: Khuyen nghi xu ly nguy co nam
{
    "ten": "N5",
    "neu": ["nguy_co_nam_benh"],
    "thi": "khuyen_nghi_phun_thuoc_nam"
},

# N6: Dat chua
{
    "ten": "N6",
    "neu": ["dat_chua"],
    "thi": "khuyen_nghi_bon_voi"
},

# N7: Thieu dam
{
    "ten": "N7",
    "neu": ["thieu_dam", "dat_vua"],
    "thi": "khuyen_nghi_bon_dam"
},

# N8: Canh bao suong muoi
{
```

```
    "ten": "N8",
    "neu": ["canh_bao_suong_muoi"],
    "thi": "khuyen_nghi_che_phu"
  }
]



# ==============================
# SUY DIEN TIEN
# ==============================


def suy_dien(su_kien, luat):
  facts = set(su_kien)
  thay_doi = True

  while thay_doi:
    thay_doi = False

    for r in luat:
      if (
        r["thi"] not in facts
        and all(dk in facts for dk in r["neu"])
      ):
        facts.add(r["thi"])
        thay_doi = True

  return facts



# ==============================
```

```
# DU LIEU CAM BIEN
# ==============================

doc_cam_bien = [
  {
    "gio": "04:00",
    "do_am_dat": 45,
    "nhiet_do": 6,
    "do_am_kk": 75,
    "ph_dat": 6.2,
    "dam": 35,
    "mua_ngay_mai": False
  },
  {
    "gio": "06:00",
    "do_am_dat": 45,
    "nhiet_do": 24,
    "do_am_kk": 90,
    "ph_dat": 6.2,
    "dam": 35,
    "mua_ngay_mai": False
  },
  {
    "gio": "10:00",
    "do_am_dat": 28,
    "nhiet_do": 33,
    "do_am_kk": 60,
    "ph_dat": 6.2,
    "dam": 35,
    "mua_ngay_mai": False
```

```
},
{
  "gio": "13:00",
  "do_am_dat": 22,
  "nhiet_do": 37,
  "do_am_kk": 45,
  "ph_dat": 6.2,
  "dam": 18,
  "mua_ngay_mai": False
},
{
  "gio": "15:00",
  "do_am_dat": 25,
  "nhiet_do": 34,
  "do_am_kk": 50,
  "ph_dat": 5.2,
  "dam": 18,
  "mua_ngay_mai": True
},
{
  "gio": "19:00",
  "do_am_dat": 80,
  "nhiet_do": 27,
  "do_am_kk": 92,
  "ph_dat": 5.2,
  "dam": 18,
  "mua_ngay_mai": True
},
{
  "gio": "22:00",
```

```
    "do_am_dat": 55,
    "nhiet_do": 25,
    "do_am_kk": 70,
    "ph_dat": 6.0,
    "dam": 15,
    "mua_ngay_mai": False
  }
]



# ==============================
# CHAY THU NGHIEM
# ==============================

for doc in doc_cam_bien:
  sk = roi_rac_hoa(doc)
  ket_qua = suy_dien(sk, LUAT)

  hanh_dong = sorted(
    f for f in ket_qua
    if f.startswith((
      "can_",
      "tuoi_",
      "hoan_",
      "khuyen_nghi_"
    ))
  )

  print(f"\n{doc['gio']} - Sự kiện: {sorted(sk)}")
  print(
```

```
    "    Hành động:",
    hanh_dong if hanh_dong else "Không cần can thiệp"
  )
```

📷 *Thu hoạch: dán ảnh kết quả sau khi cải tiến*



```
    "    Hành động:",
    hanh_dong if hanh_dong else "Không cần can thiệp"
  )
```

```
...
    04:00 - Sự kiện: ['canh_bao_suong_muoi', 'dat_vua', 'khong_co_mua']
         Hành động: ['khuyen_nghi_che_phu']

    06:00 - Sự kiện: ['dat_vua', 'khong_co_mua', 'khong_khi_am']
         Hành động: Không cần can thiệp

    10:00 - Sự kiện: ['dat_kho', 'khong_co_mua']
         Hành động: ['can_tuoi']

    13:00 - Sự kiện: ['dat_kho', 'khong_co_mua', 'nong', 'thieu_dam']
         Hành động: ['can_tuoi', 'tuoi_ngay_lap_tuc']

    15:00 - Sự kiện: ['co_mua', 'dat_chua', 'dat_kho', 'thieu_dam']
         Hành động: ['hoan_tuoi_cho_mua', 'khuyen_nghi_bon_voi']

    19:00 - Sự kiện: ['co_mua', 'dat_chua', 'dat_uot', 'khong_khi_am', 'thieu_dam']
         Hành động: ['khuyen_nghi_bon_voi', 'khuyen_nghi_phun_thuoc_nam']

    22:00 - Sự kiện: ['dat_vua', 'khong_co_mua', 'thieu_dam']
         Hành động: ['khuyen_nghi_bon_dam']
```

Bài 8 — Tài chính: chấm điểm rủi ro giao dịch bằng hệ luật có trọng số và giải thích quyết định

Liên hệ đa lĩnh vực (Tài chính / Ngân hàng): hệ thống chống gian lận thường kết hợp nhiều luật nhỏ, mỗi luật cộng một số điểm rủi ro, rồi đối chiếu ngưỡng để cho qua, yêu cầu xác thực bổ sung hay chặn. Ưu điểm của hệ luật so với hộp đen là giải thích được vì sao giao dịch bị chặn (yêu cầu pháp lý và chăm sóc khách hàng). (Giao dịch giả lập.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
   # ---- Chấm điểm rủi ro giao dịch ngân hàng bằng hệ luật có trọng số, có giải thích
LUAT_RUI_RO = [
     {"ten": "F1", "mo_ta": "Số tiền > 5 lần mức giao dịch trung bình",       "dk": lambda g:
g["so_tien"] > 5 * g["tb_30_ngay"],         "diem": 30},
     {"ten": "F2", "mo_ta": "Giao dịch từ quốc gia khác quốc gia cư trú",      "dk": lambda g:
g["quoc_gia"] != g["quoc_gia_cu_tru"],    "diem": 25},
     {"ten": "F3", "mo_ta": "Giao dịch ban đêm (0h-5h)",                        "dk": lambda g: 0
<= g["gio"] < 5,                          "diem": 15},
     {"ten": "F4", "mo_ta": "Thiết bị lạ (chưa từng dùng)",                   "dk": lambda g: not
g["thiet_bi_quen"],                    "diem": 20},
     {"ten": "F5", "mo_ta": "Nhập sai OTP từ 2 lần trở lên",                    "dk": lambda g:
g["otp_sai"] >= 2,                        "diem": 30},
     {"ten": "F6", "mo_ta": "Số tiền sát hạn mức (95%-100%) để né kiểm soát", "dk": lambda g:
0.95 * g["han_muc"] <= g["so_tien"] <= g["han_muc"], "diem": 25},
]
NGUONG_CHAN, NGUONG_OTP = 70, 40
```

```
def danh_gia(g, luat=LUAT_RUI_RO):
    kich_hoat = [r for r in luat if r["dk"](g)]
    diem = sum(r["diem"] for r in kich_hoat)
     quyet_dinh = "CHẶN" if diem >= NGUONG_CHAN else "YÊU CẦU OTP BỔ SUNG" if diem >= NGUONG_OTP
else "CHO QUA"
    return diem, quyet_dinh, kich_hoat

def   gd(so_tien,   gio=14,   quoc_gia="VN",   thiet_bi_quen=True,   otp_sai=0,   tb=2_000_000,
han_muc=50_000_000):
    return dict(so_tien=so_tien, gio=gio, quoc_gia=quoc_gia, quoc_gia_cu_tru="VN",
                thiet_bi_quen=thiet_bi_quen, otp_sai=otp_sai, tb_30_ngay=tb, han_muc=han_muc)

# (giao dịch, nhãn thật: có phải gian lận không)
mau = [
    ("Mua cà phê",                      gd(45_000), 0),
    ("Chuyển tiền thuê nhà",             gd(8_000_000), 0),
    ("Mua hàng nước ngoài ban ngày",    gd(3_000_000, quoc_gia="SG"), 0),
    ("Rút gần hết hạn mức",              gd(49_000_000), 1),
    ("Chuyển lớn, đêm, thiết bị lạ",    gd(30_000_000, gio=2, thiet_bi_quen=False), 1),
    ("Nước ngoài + đêm + OTP sai",      gd(12_000_000, gio=3, quoc_gia="RU", otp_sai=3), 1),
    ("Đổi điện thoại mới, chuyển nhỏ",  gd(500_000, thiet_bi_quen=False), 0),
    ("Chuyển lớn bất thường ban ngày",  gd(15_000_000), 0),
]

print(f"{'giao dịch':<34}{'điểm':>6}  {'quyết định':<22}{'luật kích hoạt'}")
tp = fp = fn = tn = 0
for ten, g, nhan in mau:
    diem, qd, kh = danh_gia(g)
    bi_chan_hoac_hoi = qd != "CHO QUA"
    tp += bi_chan_hoac_hoi and nhan == 1
    fp += bi_chan_hoac_hoi and nhan == 0
    fn += (not bi_chan_hoac_hoi) and nhan == 1
    tn += (not bi_chan_hoac_hoi) and nhan == 0
    print(f"{ten:<34}{diem:>6}  {qd:<22}{[r['ten'] for r in kh]}")

print(f"\nBắt được {tp}/{tp + fn} giao dịch gian lận; cảnh báo nhầm {fp}/{fp + tn} giao dịch
bình thường")
```

```
print(f"\nBắt được {tp}/{tp + fn} giao dịch gian lận; cảnh báo nhầm {fp}/{fp + tn} giao dịch bình thường")
```

```
...  giao dịch                          điểm  quyết định            luật kích hoạt
     Mua cà phê                            0  CHO QUA               []
     Chuyển tiền thuê nhà                  0  CHO QUA               []
     Mua hàng nước ngoài ban ngày         25  CHO QUA               ['F2']
     Rút gần hết hạn mức                  55  YÊU CẦU OTP BỔ SUNG   ['F1', 'F6']
     Chuyển lớn, đêm, thiết bị lạ         65  YÊU CẦU OTP BỔ SUNG   ['F1', 'F3', 'F4']
     Nước ngoài + đêm + OTP sai          100  CHẶN                  ['F1', 'F2', 'F3', 'F5']
     Đổi điện thoại mới, chuyển nhỏ       20  CHO QUA               ['F4']
     Chuyển lớn bất thường ban ngày       30  CHO QUA               ['F1']

     Bắt được 3/3 giao dịch gian lận; cảnh báo nhầm 0/5 giao dịch bình thường
```

💭 **Suy nghĩ: Giao dịch "Chuyển lớn bất thường ban ngày" chỉ bị kích hoạt luật F1 và được cho qua — đó là lựa chọn hợp lý hay có thể là kẽ hở? Vì sao hệ thống có mức trung gian "yêu cầu OTP bổ sung" thay vì chỉ có chặn hoặc cho qua? Việc in ra danh sách luật kích hoạt có giá trị gì với nhân viên chăm sóc khách hàng?**

Giao dịch “Chuyển lớn bất thường ban ngày” được 30 điểm từ F1 và được cho qua. Đây vừa có thể là lựa chọn hợp lý, vừa tiềm ẩn kẽ hở. Hợp lý vì một giao dịch có số tiền lớn hơn mức trung bình chưa chắc là gian lận; khách hàng có thể đang thanh toán học phí, mua xe hoặc chuyển tiền hợp pháp. Tuy nhiên, đây cũng là kẽ hở nếu kẻ gian lợi dụng giao dịch vào ban ngày, dùng thiết bị quen thuộc và không nhập sai OTP để tránh kích hoạt thêm các luật rủi ro. Vì vậy, hệ thống nên theo dõi lịch sử giao dịch, người nhận tiền và hành vi bất thường thay vì chỉ dựa vào một yếu tố.

Hệ thống có mức trung gian “YÊU CẦU OTP BỔ SUNG” vì không phải giao dịch nào có dấu hiệu bất thường cũng cần bị chặn ngay. Nếu chỉ có hai lựa chọn là chặn hoặc cho qua, hệ thống có thể gây bất tiện cho khách hàng hợp pháp hoặc bỏ lọt giao dịch đáng ngờ. OTP bổ sung tạo ra một bước xác minh danh tính trước khi hoàn tất giao dịch. Ví dụ, giao dịch đạt 55 hoặc 65 điểm sẽ được yêu cầu xác thực thêm thay vì bị chặn ngay. Điều này giúp cân bằng giữa bảo mật và trải nghiệm khách hàng.

Việc in ra danh sách luật kích hoạt có giá trị lớn đối với nhân viên chăm sóc khách hàng vì giúp họ biết vì sao giao dịch bị cảnh báo. Chẳng hạn, nếu giao dịch kích hoạt F1, F3 và F4, nhân viên có thể giải thích rằng giao dịch có số tiền bất thường, được thực hiện vào ban đêm và sử dụng thiết bị lạ. Nhờ đó, nhân viên có căn cứ để tư vấn khách hàng xác minh giao dịch, hỗ trợ xử lý khiếu nại và chuyển trường hợp nghi ngờ đến bộ phận an ninh ngân hàng. Danh sách này cũng giúp ngân hàng kiểm tra lại quyết định của hệ thống và phát hiện những luật đang hoạt động chưa hợp lý.

Tóm lại: Hệ thống không nên chỉ tập trung vào việc chặn giao dịch mà cần phân loại mức độ rủi ro, xác minh bổ sung và giải thích rõ nguyên nhân để vừa hạn chế gian lận vừa tránh ảnh hưởng đến khách hàng hợp pháp.

🔧 Cải tiến: Thử các cặp ngưỡng (NGUONG_CHAN, NGUONG_OTP) = (60, 35), (80, 50) và (50, 30); ghi lại số gian lận bắt được và số cảnh báo nhầm của từng cặp rồi chọn cặp bạn thấy hợp lý. Sau đó thêm luật F7 "giao dịch đến tài khoản người nhận mới" (thêm trường nguoi_nhan_moi vào dữ liệu) với trọng số tự chọn.

```
LUAT_RUI_RO = [
  {"ten": "F1", "mo_ta": "Số tiền > 5 lần mức trung bình",
   "dk": lambda g: g["so_tien"] > 5 * g["tb_30_ngay"], "diem": 30},

  {"ten": "F2", "mo_ta": "Giao dịch từ quốc gia khác",
   "dk": lambda g: g["quoc_gia"] != g["quoc_gia_cu_tru"], "diem": 25},

  {"ten": "F3", "mo_ta": "Giao dịch ban đêm (0h-5h)",
   "dk": lambda g: 0 <= g["gio"] < 5, "diem": 15},

  {"ten": "F4", "mo_ta": "Thiết bị lạ",
   "dk": lambda g: not g["thiet_bi_quen"], "diem": 20},

  {"ten": "F5", "mo_ta": "Nhập sai OTP từ 2 lần",
   "dk": lambda g: g["otp_sai"] >= 2, "diem": 30},

  {"ten": "F6", "mo_ta": "Số tiền sát hạn mức",
   "dk": lambda g: 0.95 * g["han_muc"] <= g["so_tien"] <= g["han_muc"],
   "diem": 25},
```

```
    {"ten": "F7", "mo_ta": "Giao dịch đến người nhận mới",
     "dk": lambda g: g["nguoi_nhan_moi"], "diem": 20},
]


# Chọn ngưỡng sau khi thử nghiệm
NGUONG_CHAN = 80
NGUONG_OTP = 50


def danh_gia(g, luat=LUAT_RUI_RO):
    kich_hoat = [r for r in luat if r["dk"](g)]
    diem = sum(r["diem"] for r in kich_hoat)

    if diem >= NGUONG_CHAN:
        quyet_dinh = "CHẶN"
    elif diem >= NGUONG_OTP:
        quyet_dinh = "YÊU CẦU OTP BỔ SUNG"
    else:
        quyet_dinh = "CHO QUA"

    return diem, quyet_dinh, kich_hoat



def gd(so_tien, gio=14, quoc_gia="VN",
       thiet_bi_quen=True, otp_sai=0,
       nguoi_nhan_moi=False,
       tb=2_000_000, han_muc=50_000_000):

    return {
        "so_tien": so_tien,
        "gio": gio,
```

```
    "quoc_gia": quoc_gia,
    "quoc_gia_cu_tru": "VN",
    "thiet_bi_quen": thiet_bi_quen,
    "otp_sai": otp_sai,
    "nguoi_nhan_moi": nguoi_nhan_moi,
    "tb_30_ngay": tb,
    "han_muc": han_muc
  }



mau = [
  ("Mua cà phê", gd(45_000), 0),

  ("Chuyển tiền thuê nhà", gd(8_000_000), 0),

  ("Mua hàng nước ngoài ban ngày",
   gd(3_000_000, quoc_gia="SG"), 0),

  ("Rút gần hết hạn mức",
   gd(49_000_000), 1),

  ("Chuyển lớn, đêm, thiết bị lạ",
   gd(30_000_000, gio=2,
     thiet_bi_quen=False, nguoi_nhan_moi=True), 1),

  ("Nước ngoài + đêm + OTP sai",
   gd(12_000_000, gio=3,
     quoc_gia="RU", otp_sai=3), 1),

  ("Đổi điện thoại mới, chuyển nhỏ",
```

```
    gd(500_000, thiet_bi_quen=False), 0),

    ("Chuyển lớn bất thường ban ngày",
     gd(15_000_000, nguoi_nhan_moi=True), 0),
]



print(f"{'GIAO DỊCH':<38}{'ĐIỂM':>6}  {'QUYẾT ĐỊNH':<25}LUẬT")
tp = fp = fn = tn = 0

for ten, g, nhan in mau:
    diem, qd, kh = danh_gia(g)

    canh_bao = qd != "CHO QUA"

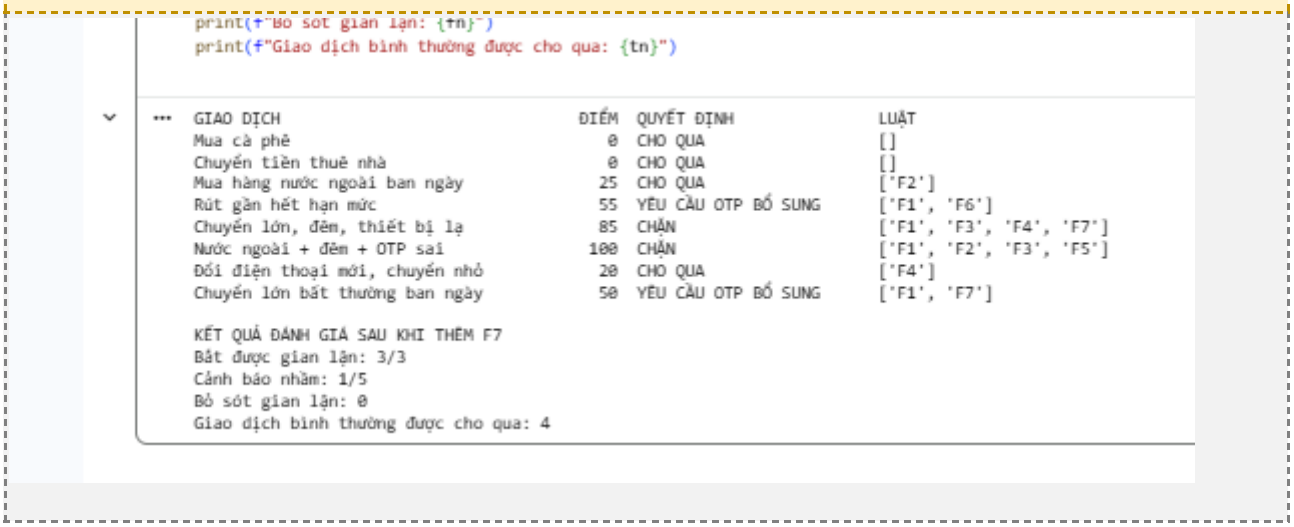
    if canh_bao and nhan == 1:
        tp += 1
    elif canh_bao and nhan == 0:
        fp += 1
    elif not canh_bao and nhan == 1:
        fn += 1
    else:
        tn += 1

    print(f"{ten:<38}{diem:>6}  {qd:<25}"
          f"{[r['ten'] for r in kh]}")

print("\nKẾT QUẢ ĐÁNH GIÁ SAU KHI THÊM F7")
print(f"Bắt được gian lận: {tp}/{tp + fn}")
print(f"Cảnh báo nhầm: {fp}/{fp + tn}")
```

print(f"Bỏ sót gian lận: {fn}")

print(f"Giao dịch bình thường được cho qua: {tn}")



```
print(f"Bỏ sót gian lận: {fn}")
print(f"Giao dịch bình thường được cho qua: {tn}")
```

GIAO DỊCH	ĐIỂM	QUYẾT ĐỊNH	LUẬT
Mua cà phê	0	CHO QUA	[]
Chuyển tiền thuê nhà	0	CHO QUA	[]
Mua hàng nước ngoài ban ngày	25	CHO QUA	['F2']
Rút gần hết hạn mức	55	YÊU CẦU OTP BỔ SUNG	['F1', 'F6']
Chuyển lớn, đêm, thiết bị lạ	85	CHẶN	['F1', 'F3', 'F4', 'F7']
Nước ngoài + đêm + OTP sai	100	CHẶN	['F1', 'F2', 'F3', 'F5']
Đổi điện thoại mới, chuyển nhỏ	20	CHO QUA	['F4']
Chuyển lớn bất thường ban ngày	50	YÊU CẦU OTP BỔ SUNG	['F1', 'F7']

```
KẾT QUẢ ĐÁNH GIÁ SAU KHI THÊM F7
Bắt được gian lận: 3/3
Cảnh báo nhầm: 1/5
Bỏ sót gian lận: 0
Giao dịch bình thường được cho qua: 4
```

Bài 9 — Giáo dục: suy diễn lùi kiểm tra điều kiện đăng ký học phần

Liên hệ đa lĩnh vực (Giáo dục): cổng đăng ký học phần phải trả lời "sinh viên này có được đăng ký môn X không, và nếu chưa thì còn thiếu gì". Suy diễn lùi đi từ mục tiêu xuống các điều kiện gốc, chỉ tra những điều kiện liên quan và liệt kê chính xác điều kiện còn thiếu. (Hồ sơ sinh viên giả lập.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
  # ---- Tư vấn học vụ: kiểm tra điều kiện đăng ký học phần bằng suy diễn lùi (đi từ mục tiêu xuống điều kiện gốc)
LUAT_HOC_VU = {
    "dang_ky_do_an_ai":      ["dang_ky_hoc_ai", "du_100_tin_chi", "khong_no_hoc_phi"],
          "dang_ky_hoc_ai":                   ["hoc_xong_lap_trinh_py",  "hoc_xong_toan_roi_rac", "du_dk_xac_suat"],
    "du_dk_xac_suat":        ["hoc_xong_giai_tich", "hoc_xong_dai_so"],
    "dang_ky_hoc_may":       ["dang_ky_hoc_ai", "hoc_xong_xac_suat_thong_ke"],
}

def chung_minh(muc_tieu, da_biet, luat, da_hoi=None, thieu=None):
     """Trả về (đạt?, danh sách điều kiện gốc còn thiếu). Chỉ xét những điều kiện liên quan đến mục tiêu."""
    if da_hoi is None:
        da_hoi, thieu = [], []
    if muc_tieu in luat:                                    # điều kiện suy ra từ các điều kiện con
        dat = True
        for con in luat[muc_tieu]:
            dat = chung_minh(con, da_biet, luat, da_hoi, thieu)[0] and dat
        return dat, thieu
    da_hoi.append(muc_tieu)                                 # điều kiện gốc: tra hồ sơ sinh viên
    if muc_tieu in da_biet:
        return True, thieu
    thieu.append(muc_tieu)
    return False, thieu

ho_so = {
         "An":       {"hoc_xong_lap_trinh_py",  "hoc_xong_toan_roi_rac",  "hoc_xong_giai_tich", "hoc_xong_dai_so",
             "du_100_tin_chi", "khong_no_hoc_phi", "hoc_xong_xac_suat_thong_ke"},
    "Binh": {"hoc_xong_lap_trinh_py", "hoc_xong_giai_tich", "du_100_tin_chi"},
          "Chi":     {"hoc_xong_lap_trinh_py",  "hoc_xong_toan_roi_rac",  "hoc_xong_giai_tich", "hoc_xong_dai_so",
             "du_100_tin_chi"},
}
```

```
so_goc = len({dk for ds in LUAT_HOC_VU.values() for dk in ds if dk not in LUAT_HOC_VU})   # tổng số điều kiện gốc trong hệ luật
for muc_tieu in ["dang_ky_do_an_ai", "dang_ky_hoc_may"]:
    print(f"##### Mục tiêu: {muc_tieu}")
    for ten, da_biet in ho_so.items():
        da_hoi = []
        dat, thieu = chung_minh(muc_tieu, da_biet, LUAT_HOC_VU, da_hoi, [])
        trang_thai = "ĐỦ điều kiện" if dat else "CHƯA đủ, còn thiếu: " + ", ".join(thieu)
        print(f"  {ten:<5} {trang_thai}   (đã tra {len(da_hoi)}/{so_goc} điều kiện gốc)")
```

```
##### Mục tiêu: dang_ky_do_an_ai
  An    ĐỦ điều kiện   (đã tra 6/7 điều kiện gốc)
  Binh  CHƯA đủ, còn thiếu: hoc_xong_toan_roi_rac, hoc_xong_dai_so, khong_no_hoc_phi   (đã tra 6/7 điều kiện gốc)
  Chi   CHƯA đủ, còn thiếu: khong_no_hoc_phi   (đã tra 6/7 điều kiện gốc)
##### Mục tiêu: dang_ky_hoc_may
  An    ĐỦ điều kiện   (đã tra 5/7 điều kiện gốc)
  Binh  CHƯA đủ, còn thiếu: hoc_xong_toan_roi_rac, hoc_xong_dai_so, hoc_xong_xac_suat_thong_ke   (đã tra 5/7 điều kiện gốc)
  Chi   CHƯA đủ, còn thiếu: hoc_xong_xac_suat_thong_ke   (đã tra 5/7 điều kiện gốc)
```

💭 **Suy nghĩ: Vì sao suy diễn lùi phù hợp hơn suy diễn tiến cho bài toán "được đăng ký môn X hay không"? Với mục tiêu dang_ky_hoc_may, vì sao hệ thống chỉ tra 5/7 điều kiện gốc? Danh sách "còn thiếu" có lợi gì cho sinh viên so với chỉ trả lời "không đủ điều kiện"?**

Vì sao suy diễn lùi phù hợp hơn suy diễn tiến?

Bài toán đăng ký học phần bắt đầu bằng một mục tiêu cụ thể, chẳng hạn: Sinh viên có đủ điều kiện đăng ký đồ án AI hay không?

Suy diễn lùi phù hợp vì hệ thống bắt đầu từ mục tiêu cần kiểm tra, sau đó truy ngược các điều kiện bắt buộc để xác minh. Ví dụ, để đăng ký đồ án AI, sinh viên phải đăng ký được học phần AI, có đủ 100 tín chỉ và không nợ học phí.

Ngược lại, suy diễn tiến sẽ bắt đầu từ toàn bộ thông tin trong hồ sơ sinh viên rồi suy ra những môn học hoặc học phần mà sinh viên có thể đăng ký. Cách này có thể phải kiểm tra nhiều điều kiện không liên quan đến mục tiêu đang xét.

Vì vậy, suy diễn lùi tiết kiệm việc tra cứu và phù hợp với bài toán xác minh một học phần cụ thể.

Vì sao mục tiêu `dang_ky_hoc_may` chỉ tra 5/7 điều kiện gốc?

Vì suy diễn lùi chỉ kiểm tra những điều kiện có liên quan đến mục tiêu đang xét.

Đối với mục tiêu `dang_ky_hoc_may`, hệ thống cần kiểm tra:

- Học xong lập trình Python.
- Học xong Toán rời rạc.
- Học xong Giải tích.
- Học xong Đại số.
- Đủ điều kiện xác suất.
- Học xong Xác suất thống kê.

Trong đó, điều kiện `du_dk_xac_suat` lại phụ thuộc vào hai điều kiện là học xong Giải tích và Đại số.

Hệ thống không cần kiểm tra `du_100_tin_chi` và `khong_no_hoc_phi` vì hai điều kiện này chỉ phục vụ mục tiêu đăng ký đồ án AI, không xuất hiện trong nhánh luật đăng ký học máy.

Do đó, hệ thống chỉ cần tra 5/7 điều kiện gốc, giúp tránh kiểm tra những thông tin không cần thiết.

Danh sách điều kiện còn thiếu có lợi ích gì?

Danh sách điều kiện còn thiếu giúp sinh viên biết chính xác nguyên nhân mình chưa đủ điều kiện đăng ký học phần, thay vì chỉ nhận thông báo chung chung là không đủ điều kiện.

Ví dụ, sinh viên Bình đăng ký đồ án AI còn thiếu:

- Học xong Toán rời rạc.
- Học xong Đại số.
- Không nợ học phí.

Nhờ vậy, sinh viên có thể biết cần hoàn thành học phần nào hoặc xử lý vấn đề học phí trước khi đăng ký lại.

Đối với nhà trường, thông tin này cũng giúp bộ phận đào tạo giải thích kết quả xét duyệt minh bạch hơn, giảm thắc mắc và hỗ trợ sinh viên lập kế hoạch học tập.

Kết luận: Suy diễn lùi giúp hệ thống tập trung kiểm tra đúng điều kiện của học phần cần đăng ký, giảm số thông tin phải tra cứu và cung cấp danh sách điều kiện còn thiếu để sinh viên biết cách khắc phục.

🔧 Cải tiến: Thêm môn mới dang_ky_hoc_sau (cần dang_ky_hoc_may và hoc_xong_thuat_toan_nang_cao) cùng điều kiện gốc mới, thêm một sinh viên mới vào ho_so và thử cả ba mục tiêu. Sau đó sửa hàm chung_minh để dừng sớm: khi một điều kiện con đã thiếu thì không tra tiếp các điều kiện còn lại của cùng luật, và so sánh số điều kiện đã tra.

```
# ---- TƯ VẤN HỌC VỤ BẰNG SUY DIỄN LÙI

LUAT_HOC_VU = {
  "dang_ky_do_an_ai": [
    "dang_ky_hoc_ai",
    "du_100_tin_chi",
    "khong_no_hoc_phi"
  ],

  "dang_ky_hoc_ai": [
    "hoc_xong_lap_trinh_py",
```

```
    "hoc_xong_toan_roi_rac",
    "du_dk_xac_suat"
  ],

  "du_dk_xac_suat": [
    "hoc_xong_giai_tich",
    "hoc_xong_dai_so"
  ],

  "dang_ky_hoc_may": [
    "dang_ky_hoc_ai",
    "hoc_xong_xac_suat_thong_ke"
  ],

  # Môn học mới
  "dang_ky_hoc_sau": [
    "dang_ky_hoc_may",
    "hoc_xong_thuat_toan_nang_cao"
  ]
}



# Hồ sơ sinh viên
ho_so = {
  "An": {
    "hoc_xong_lap_trinh_py",
    "hoc_xong_toan_roi_rac",
    "hoc_xong_giai_tich",
    "hoc_xong_dai_so",
    "du_100_tin_chi",
```

```
    "khong_no_hoc_phi",
    "hoc_xong_xac_suat_thong_ke"
  },

  "Binh": {
    "hoc_xong_lap_trinh_py",
    "hoc_xong_giai_tich",
    "du_100_tin_chi"
  },

  "Chi": {
    "hoc_xong_lap_trinh_py",
    "hoc_xong_toan_roi_rac",
    "hoc_xong_giai_tich",
    "hoc_xong_dai_so",
    "du_100_tin_chi"
  },

  # Sinh viên mới: đã hoàn thành tất cả điều kiện
  "Dung": {
    "hoc_xong_lap_trinh_py",
    "hoc_xong_toan_roi_rac",
    "hoc_xong_giai_tich",
    "hoc_xong_dai_so",
    "du_100_tin_chi",
    "khong_no_hoc_phi",
    "hoc_xong_xac_suat_thong_ke",
    "hoc_xong_thuat_toan_nang_cao"
  }
}
```

```
# Phiên bản cũ: kiểm tra tất cả điều kiện
def chung_minh_day_du(muc_tieu, da_biet, luat,
                      da_hoi=None, thieu=None):

    if da_hoi is None:
        da_hoi, thieu = [], []

    if muc_tieu in luat:
        dat = True

        for con in luat[muc_tieu]:
            dat_con, _ = chung_minh_day_du(
                con, da_biet, luat, da_hoi, thieu
            )
            dat = dat_con and dat

        return dat, thieu

    da_hoi.append(muc_tieu)

    if muc_tieu in da_biet:
        return True, thieu

    thieu.append(muc_tieu)
    return False, thieu


# Phiên bản mới: gặp điều kiện thiếu thì dừng sớm
```

```
def chung_minh_dung_som(muc_tieu, da_biet, luat,
                        da_hoi=None, thieu=None):

    if da_hoi is None:
        da_hoi, thieu = [], []

    if muc_tieu in luat:
        for con in luat[muc_tieu]:

            dat_con, _ = chung_minh_dung_som(
                con, da_biet, luat, da_hoi, thieu
            )

            # Nếu một điều kiện con không đạt thì dừng
            if not dat_con:
                return False, thieu

        return True, thieu

    # Điều kiện gốc: tra hồ sơ sinh viên
    da_hoi.append(muc_tieu)

    if muc_tieu in da_biet:
        return True, thieu

    thieu.append(muc_tieu)
    return False, thieu



# Tổng số điều kiện gốc
```

```
so_goc = len({
    dk
    for ds in LUAT_HOC_VU.values()
    for dk in ds
    if dk not in LUAT_HOC_VU
})

muc_tieu_list = [
    "dang_ky_do_an_ai",
    "dang_ky_hoc_may",
    "dang_ky_hoc_sau"
]

print("TỔNG SỐ ĐIỀU KIỆN GỐC:", so_goc)

for muc_tieu in muc_tieu_list:
    print("\n##### MỤC TIÊU:", muc_tieu)

    for ten, da_biet in ho_so.items():

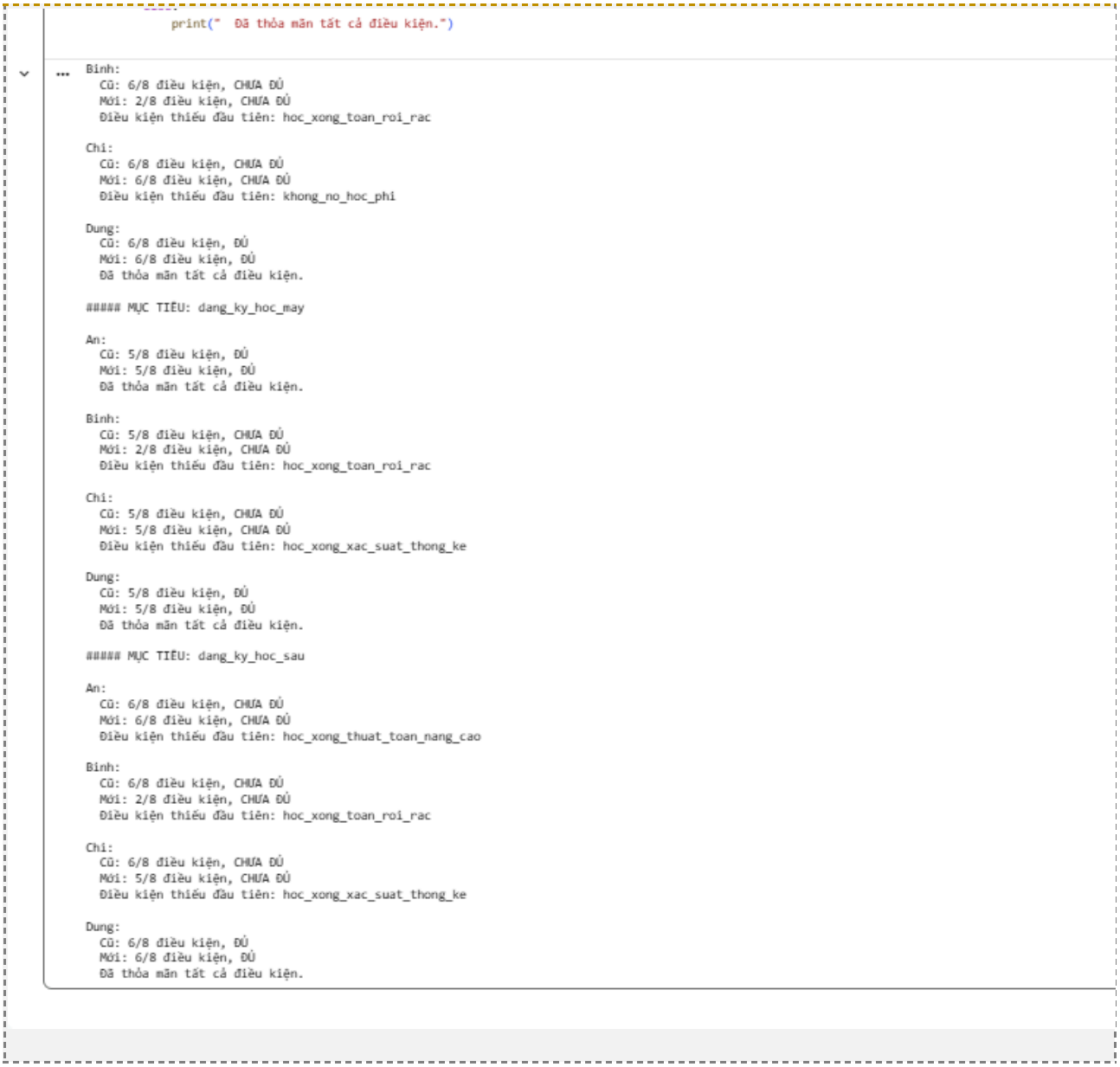
        # Chạy phiên bản cũ
        da_hoi_cu = []
        dat_cu, thieu_cu = chung_minh_day_du(
            muc_tieu, da_biet, LUAT_HOC_VU,
            da_hoi_cu, []
        )

        # Chạy phiên bản mới
        da_hoi_moi = []
        dat_moi, thieu_moi = chung_minh_dung_som(
```

```
    muc_tieu, da_biet, LUAT_HOC_VU,
    da_hoi_moi, []
)

print(f"\n{ten}:")
print(
    f"  Cũ: {len(da_hoi_cu)}/{so_goc} điều kiện, "
    f"{'ĐỦ' if dat_cu else 'CHƯA ĐỦ'}"
)
print(
    f"  Mới: {len(da_hoi_moi)}/{so_goc} điều kiện, "
    f"{'ĐỦ' if dat_moi else 'CHƯA ĐỦ'}"
)

if not dat_moi:
    print("  Điều kiện thiếu đầu tiên:", thieu_moi[0])
else:
    print("  Đã thỏa mãn tất cả điều kiện.")
```



```
            print("  Đã thỏa mãn tất cả điều kiện.")
```

```
Binh:
  Cũ: 6/8 điều kiện, CHƯA ĐỦ
  Mới: 2/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: hoc_xong_toan_roi_rac

Chi:
  Cũ: 6/8 điều kiện, CHƯA ĐỦ
  Mới: 6/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: khong_no_hoc_phi

Dung:
  Cũ: 6/8 điều kiện, ĐỦ
  Mới: 6/8 điều kiện, ĐỦ
  Đã thỏa mãn tất cả điều kiện.

##### MỤC TIÊU: dang_ky_hoc_may

An:
  Cũ: 5/8 điều kiện, ĐỦ
  Mới: 5/8 điều kiện, ĐỦ
  Đã thỏa mãn tất cả điều kiện.

Binh:
  Cũ: 5/8 điều kiện, CHƯA ĐỦ
  Mới: 2/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: hoc_xong_toan_roi_rac

Chi:
  Cũ: 5/8 điều kiện, CHƯA ĐỦ
  Mới: 5/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: hoc_xong_xac_suat_thong_ke

Dung:
  Cũ: 5/8 điều kiện, ĐỦ
  Mới: 5/8 điều kiện, ĐỦ
  Đã thỏa mãn tất cả điều kiện.

##### MỤC TIÊU: dang_ky_hoc_sau

An:
  Cũ: 6/8 điều kiện, CHƯA ĐỦ
  Mới: 6/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: hoc_xong_thuat_toan_nang_cao

Binh:
  Cũ: 6/8 điều kiện, CHƯA ĐỦ
  Mới: 2/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: hoc_xong_toan_roi_rac

Chi:
  Cũ: 6/8 điều kiện, CHƯA ĐỦ
  Mới: 5/8 điều kiện, CHƯA ĐỦ
  Điều kiện thiếu đầu tiên: hoc_xong_xac_suat_thong_ke

Dung:
  Cũ: 6/8 điều kiện, ĐỦ
  Mới: 6/8 điều kiện, ĐỦ
  Đã thỏa mãn tất cả điều kiện.
```

Bài 10 — Động cơ suy diễn dùng chung: tách tri thức (JSON) khỏi mã nguồn để áp dụng cho mọi lĩnh vực

Liên hệ CNTT và đa lĩnh vực: thay vì viết lại chương trình cho mỗi lĩnh vực, kỹ sư tách động cơ suy diễn (code) khỏi bộ luật (dữ liệu JSON do chuyên gia lĩnh vực cung cấp). Bài này có kiểm tra tính hợp lệ của bộ luật trước khi chạy, đúng như một hệ thống thực tế phải làm khi nhận dữ liệu từ người khác.

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import json

# ---- Tri thức (luật) tách khỏi mã nguồn: lưu dạng JSON, thay đổi lĩnh vực chỉ cần thay dữ liệu
KB_NHA_THONG_MINH = """
{"linh_vuc": "nha_thong_minh",
 "luat": [
   {"neu": ["khong_co_nguoi", "den_dang_bat"],            "thi": "tat_den"},
   {"neu": ["nhiet_do_cao", "co_nguoi"],                  "thi": "bat_dieu_hoa"},
   {"neu": ["cua_mo", "ban_dem"],                         "thi": "canh_bao_an_ninh"},
   {"neu": ["canh_bao_an_ninh", "khong_co_chu_nha"],      "thi": "gui_thong_bao_dien_thoai"}
 ]}
"""
KB_TUYEN_DUNG = """
{"linh_vuc": "tuyen_dung",
 "luat": [
```

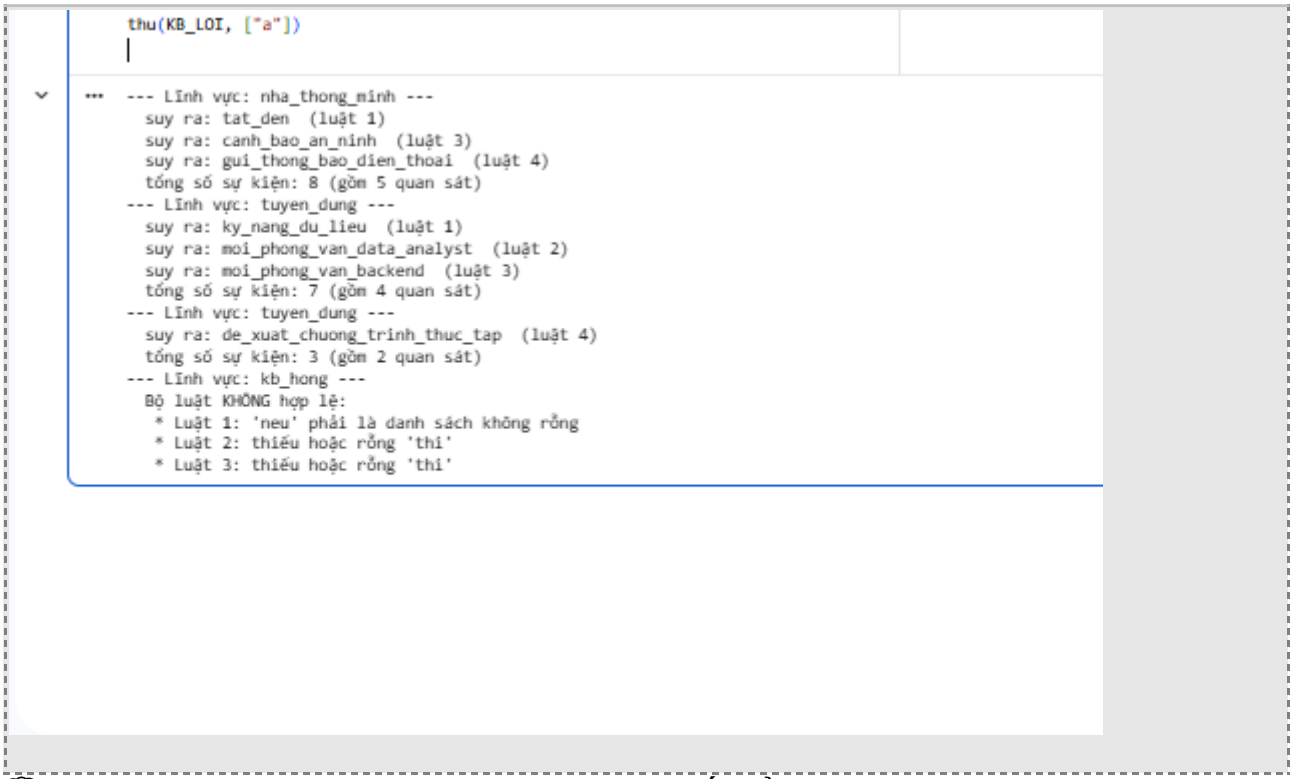
```
  {"neu": ["biet_python", "biet_sql"],                    "thi": "ky_nang_du_lieu"},
  {"neu": ["ky_nang_du_lieu", "kinh_nghiem_2_nam"],       "thi": "moi_phong_van_data_analyst"},
  {"neu": ["biet_python", "kinh_nghiem_2_nam", "biet_docker"], "thi": "moi_phong_van_backend"},
  {"neu": ["chua_co_kinh_nghiem", "biet_python"],         "thi": "de_xuat_chuong_trinh_thuc_tap"}
 ]}
"""
KB_LOI = """
{"linh_vuc": "kb_hong",
 "luat": [
  {"neu": "bat_den",             "thi": "tat_den"},
  {"neu": ["a"],                 "thi": ""},
  {"neu": ["a"]}
 ]}
"""

def kiem_tra_kb(kb):
    """Trả về danh sách lỗi (rỗng nếu hợp lệ)"""
    loi = []
    if "luat" not in kb or not isinstance(kb["luat"], list):
        return ["Thiếu danh sách 'luat'"]
    for i, r in enumerate(kb["luat"], 1):
        if not isinstance(r.get("neu"), list) or not r.get("neu"):
            loi.append(f"Luật {i}: 'neu' phải là danh sách không rỗng")
        if not isinstance(r.get("thi"), str) or not r.get("thi"):
            loi.append(f"Luật {i}: thiếu hoặc rỗng 'thi'")
    return loi

def chay(kb, su_kien):
    facts = {f: "quan sát" for f in su_kien}
    thay_doi = True
    while thay_doi:
        thay_doi = False
        for i, r in enumerate(kb["luat"], 1):
            if r["thi"] not in facts and all(dk in facts for dk in r["neu"]):
                facts[r["thi"]] = f"luật {i}"
                thay_doi = True
    return facts

def thu(kb_json, su_kien):
    kb = json.loads(kb_json)
    loi = kiem_tra_kb(kb)
    print(f"--- Lĩnh vực: {kb['linh_vuc']} ---")
    if loi:
        print("  Bộ luật KHÔNG hợp lệ:")
        for l in loi:
            print("   *", l)
        return
    facts = chay(kb, su_kien)
    for f, ly_do in facts.items():
        if ly_do != "quan sát":
            print(f"  suy ra: {f}  ({ly_do})")
    print(f"  tổng số sự kiện: {len(facts)} (gồm {len(su_kien)} quan sát)")

thu(KB_NHA_THONG_MINH,      ["khong_co_nguoi",      "den_dang_bat",      "cua_mo",      "ban_dem",
"khong_co_chu_nha"])
thu(KB_TUYEN_DUNG, ["biet_python", "biet_sql", "kinh_nghiem_2_nam", "biet_docker"])
thu(KB_TUYEN_DUNG, ["biet_python", "chua_co_kinh_nghiem"])
thu(KB_LOI, ["a"])
```



💭 **Suy nghĩ: Lợi ích của việc tách luật ra file JSON so với viết thẳng luật trong code là gì (gợi ý: ai sửa luật, có cần triển khai lại không)? Vì sao phải kiểm tra bộ luật hỏng trước khi chạy suy diễn, và nếu không kiểm tra thì có thể xảy ra điều gì? Cùng một động cơ chạy được cả nhà thông minh lẫn tuyển dụng nói lên điều gì về tính tổng quát?**

Việc tách luật ra file JSON giúp hệ chuyên gia linh hoạt và dễ bảo trì hơn so với việc viết trực tiếp các luật trong mã nguồn. Cụ thể:

Thứ nhất, lợi ích của việc tách luật ra JSON: Luật và chương trình xử lý được tách biệt. Người có chuyên môn nghiệp vụ, chẳng hạn chuyên gia nhà thông minh hoặc nhân viên tuyển dụng, có thể thay đổi các điều kiện và kết luận trong JSON mà không cần sửa thuật toán suy diễn. Khi luật thay đổi, thông thường chỉ cần cập nhật file JSON, không cần viết lại hoặc triển khai lại mã nguồn, miễn là cấu trúc dữ liệu vẫn tương thích với chương trình.

Thứ hai, vì sao phải kiểm tra bộ luật trước khi suy diễn: Việc kiểm tra giúp phát hiện các lỗi như thiếu điều kiện `neu`, `neu` không phải danh sách, thiếu kết luận `thi` hoặc kết luận bị để trống. Nếu không kiểm tra, chương trình có thể gặp lỗi khi truy cập dữ liệu, chẳng hạn `KeyError`, `TypeError`, hoặc suy diễn sai kết quả. Ví dụ, trong `KB_LOI`, luật 1 có `neu` là chuỗi thay vì danh sách, luật 2 thiếu kết luận và luật 3 thiếu trường `thi`. Hàm `kiem_tra_kb()` phát hiện những lỗi này trước khi chương trình chạy, giúp hệ thống an toàn và dễ bảo trì hơn.

Thứ ba, ý nghĩa về tính tổng quát: Cùng một động cơ suy diễn có thể chạy cho cả lĩnh vực nhà thông minh và tuyển dụng chỉ bằng cách thay đổi bộ luật JSON và dữ kiện đầu vào. Điều này cho thấy thuật toán suy diễn không phụ thuộc vào một lĩnh vực cụ thể mà có thể tái sử dụng cho nhiều bài toán khác nhau. Đây là đặc điểm quan trọng của hệ chuyên gia: tách biệt cơ sở tri thức và động cơ suy diễn.

Kết luận: Tách luật khỏi mã nguồn giúp dễ cập nhật, giảm phụ thuộc vào lập trình viên, kiểm soát lỗi tốt hơn và cho phép tái sử dụng cùng một động cơ suy diễn trong nhiều lĩnh vực.

🔧 Cải tiến: Viết thêm một bộ luật JSON cho một lĩnh vực bạn chọn (du lịch, thư viện, bán lẻ, trường học...) với ít nhất 5 luật và chạy bằng đúng hàm thu(). Sau đó thêm kiểm tra phát hiện luật có điều kiện trùng lặp và luật có kết luận nằm trong chính điều kiện của nó (tự suy diễn vô nghĩa), rồi thử với một bộ luật cố ý viết sai.

```
import json

# ---- 1. Bộ luật JSON quản lý thư viện

KB_THU_VIEN = """
{
  "linh_vuc": "quan_ly_thu_vien",
  "luat": [
    {
      "neu": ["co_the_thu_vien", "sach_con_hang"],
      "thi": "du_dieu_kien_muon"
    },
    {
      "neu": ["du_dieu_kien_muon", "khong_no_sach"],
      "thi": "cho_phep_muon"
    },
    {
      "neu": ["dang_muon_toi_da"],
      "thi": "tu_choi_muon"
    },
    {
      "neu": ["sach_qua_han"],
      "thi": "canh_bao_qua_han"
    },
    {
      "neu": ["canh_bao_qua_han", "co_email"],
```

```
      "thi": "gui_email_nhac_nho"
    },
    {
      "neu": ["sach_hu_hong"],
      "thi": "yeu_cau_boi_thuong"
    }
  ]
}
"""


# ---- 2. Bộ luật cố ý viết sai để kiểm thử

KB_THU_VIEN_LOI = """
{
  "linh_vuc": "thu_vien_loi",
  "luat": [
    {
      "neu": ["co_the_thu_vien", "sach_con_hang"],
      "thi": "cho_phep_muon"
    },
    {
      "neu": ["sach_con_hang", "co_the_thu_vien"],
      "thi": "du_dieu_kien_muon"
    },
    {
      "neu": ["sach_qua_han"],
      "thi": "sach_qua_han"
    },
    {
```

```
        "neu": ["co_the_thu_vien", "co_the_thu_vien"],
        "thi": "du_dieu_kien_muon"
    }
  ]
}
"""

# ---- 3. Kiểm tra bộ luật

def kiem_tra_kb(kb):
  loi = []

  if "luat" not in kb or not isinstance(kb["luat"], list):
    return ["Thiếu danh sách 'luat'"]

  # Lưu điều kiện của các luật đã gặp
  dieu_kien_da_gap = {}

  for i, r in enumerate(kb["luat"], 1):

    if not isinstance(r, dict):
      loi.append(f"Luật {i}: cấu trúc luật không hợp lệ")
      continue

    neu = r.get("neu")
    thi = r.get("thi")

    # Kiểm tra neu
    if not isinstance(neu, list) or not neu:
```

```
        loi.append(
            f"Luật {i}: 'neu' phải là danh sách không rỗng"
        )
        continue

    if not all(isinstance(x, str) and x.strip() for x in neu):
        loi.append(
            f"Luật {i}: điều kiện phải là chuỗi không rỗng"
        )
        continue

    # Kiểm tra thi
    if not isinstance(thi, str) or not thi.strip():
        loi.append(f"Luật {i}: thiếu hoặc rỗng 'thi'")
        continue

    # Kiểm tra điều kiện bị lặp trong cùng luật
    if len(neu) != len(set(neu)):
        loi.append(
            f"Luật {i}: có điều kiện bị lặp trong chính luật"
        )

    # Kiểm tra điều kiện trùng với luật khác
    khoa = frozenset(neu)

    if khoa in dieu_kien_da_gap:
        luat_truoc = dieu_kien_da_gap[khoa]
        loi.append(
            f"Luật {i}: trùng tập điều kiện với luật {luat_truoc}"
        )
```

```
        else:
            dieu_kien_da_gap[khoa] = i

        # Kiểm tra luật tự suy diễn
        if thi in neu:
            loi.append(
                f"Luật {i}: kết luận '{thi}' nằm trong chính điều kiện"
            )

    return loi


# ---- 4. Động cơ suy diễn

def chay(kb, su_kien):
    facts = {f: "quan sát" for f in su_kien}

    thay_doi = True

    while thay_doi:
        thay_doi = False

        for i, r in enumerate(kb["luat"], 1):
            if (
                r["thi"] not in facts
                and all(dk in facts for dk in r["neu"])
            ):
                facts[r["thi"]] = f"luật {i}"
                thay_doi = True
```

```
    return facts



# ---- 5. Hàm thu() giữ cách sử dụng như ban đầu


def thu(kb_json, su_kien):
    try:
        kb = json.loads(kb_json)
    except json.JSONDecodeError as e:
        print("--- JSON KHÔNG HỢP LỆ ---")
        print("Lỗi cú pháp:", e)
        return

    if not isinstance(kb, dict):
        print("--- Bộ luật không hợp lệ: dữ liệu gốc phải là object ---")
        return

    print(f"--- Lĩnh vực: {kb.get('linh_vuc', 'chưa xác định')} ---")

    loi = kiem_tra_kb(kb)

    if loi:
        print("Bộ luật KHÔNG hợp lệ:")
        for l in loi:
            print(" *", l)
        return

    facts = chay(kb, su_kien)

    for f, ly_do in facts.items():
```

```
        if ly_do != "quan sát":
            print(f"  suy ra: {f} ({ly_do})")

    print(
        f"  Tổng số sự kiện: {len(facts)} "
        f"(gồm {len(su_kien)} quan sát)"
    )


# ---- 6. Chạy thử bộ luật thư viện

print("===== KIỂM THỬ BỘ LUẬT THƯ VIỆN =====")

thu(
    KB_THU_VIEN,
    [
        "co_the_thu_vien",
        "sach_con_hang",
        "khong_no_sach",
        "sach_qua_han",
        "co_email"
    ]
)


# ---- 7. Kiểm thử bộ luật cố ý viết sai

print("\n===== KIỂM THỬ BỘ LUẬT CỐ Ý VIẾT SAI =====")

thu(
```

```
KB_THU_VIEN_LOI,
["co_the_thu_vien", "sach_con_hang"]
)
```

```
    )

===== KIỂM THỬ BỘ LUẬT THƯ VIỆN =====
--- Lĩnh vực: quan_ly_thu_vien ---
  suy ra: du_dieu_kien_muon (luật 1)
  suy ra: cho_phep_muon (luật 2)
  suy ra: canh_bao_qua_han (luật 4)
  suy ra: gui_email_nhac_nho (luật 5)
  Tổng số sự kiện: 9 (gồm 5 quan sát)

===== KIỂM THỬ BỘ LUẬT CỐ Ý VIẾT SAI =====
--- Lĩnh vực: thu_vien_loi ---
Bộ luật KHÔNG hợp lệ:
 * Luật 2: trùng tập điều kiện với luật 1
 * Luật 3: kết luận 'sach_qua_han' nằm trong chính điều kiện
 * Luật 4: có điều kiện bị lặp trong chính luật
```

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Tôi đã thực hành xây dựng và chạy các chương trình hệ chuyên gia bằng Python, bao gồm chẩn đoán lỗi mạng, tư vấn chọn ngành học, kiểm tra điều kiện đăng ký học phần bằng suy diễn tiến và suy diễn lùi, chấm điểm rủi ro giao dịch ngân hàng và quản lý thư viện bằng luật JSON. Kết quả cho thấy hệ thống có thể sử dụng các dữ kiện đầu vào để kích hoạt những luật IF-THEN, từ đó đưa ra kết luận hoặc đề xuất phù hợp. Tôi cũng thử nghiệm việc thêm luật mới, thay đổi ngưỡng đánh giá và kiểm tra lỗi trong cơ sở tri thức.

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Nhìn chung, kết quả tương đối giống dự đoán ban đầu. Tuy nhiên, điều khiến tôi bất ngờ là hệ thống có thể suy diễn qua nhiều bước, sử dụng kết luận trước đó làm dữ kiện để tiếp tục kích hoạt luật tiếp theo. Tôi cũng nhận thấy suy diễn lùi có thể giảm số lần kiểm tra nhờ chỉ tập trung vào mục tiêu cần xác minh. Ngoài ra, việc thay đổi trọng số và ngưỡng trong bài toán phát hiện gian lận có thể làm thay đổi quyết định của hệ thống, cho thấy việc xây dựng luật và lựa chọn điều kiện rất quan trọng.

3. KHÁI QUÁT HÓA — Cơ sở tri thức và luật suy diễn là gì? Forward Chaining khác Backward Chaining như thế nào? (diễn đạt bằng lời của chính mình)

Theo cách hiểu của tôi, cơ sở tri thức là nơi lưu trữ các thông tin, dữ kiện và quy tắc mà hệ chuyên gia sử dụng để giải quyết một bài toán. Luật suy diễn thường được biểu diễn dưới dạng IF-THEN, nghĩa là nếu các điều kiện được thỏa mãn thì hệ thống có thể đưa ra một kết luận tương ứng. Forward Chaining (suy diễn tiến) bắt đầu từ những dữ kiện đã có rồi áp dụng các luật để tìm ra kết luận mới. Ngược lại, Backward Chaining (suy diễn lùi) bắt đầu từ một mục tiêu cần chứng minh, sau đó kiểm tra ngược lại những điều kiện cần thiết để xác định mục tiêu đó có đúng hay không.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu muốn xây một hệ chuyên gia cho một lĩnh vực mới, tôi sẽ bắt đầu thu thập luật như thế nào? Tôi còn thắc mắc điều gì?

Nếu xây dựng hệ chuyên gia cho một lĩnh vực mới, tôi sẽ bắt đầu bằng việc xác định bài toán cần giải quyết, sau đó tìm hiểu quy trình thực tế và thu thập kiến thức từ chuyên gia hoặc tài liệu chuyên ngành. Tiếp theo, tôi sẽ chuyển những kiến thức đó thành các luật IF-THEN, xây dựng cơ sở tri thức, kiểm tra tính hợp lệ của luật và thử nghiệm bằng nhiều bộ dữ liệu khác nhau. Qua buổi thực hành, tôi nhận thấy cần chú ý đến tính chính xác, tính đầy đủ và khả năng giải thích của hệ thống. Điều tôi còn muốn tìm hiểu thêm là làm thế nào để hệ chuyên gia có thể tự cập nhật kiến thức, xử lý những trường hợp không chắc chắn và đưa ra kết luận đáng tin cậy khi dữ liệu đầu vào chưa đầy đủ.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — 5

✅ Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ code cải tiến, ảnh và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.

123000609-Nguyenquachphutai.ipynb